

World Models for Embodied AI: Manipulation, Locomotion and Navigation

SEMINAR REPORT

Submitted by

Navaneeth K P

SNM23CA013

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

in

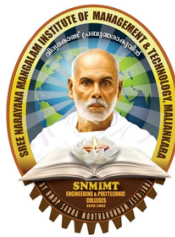
Computer Science and Engineering (Artificial Intelligence)

Under the guidance of

Ms. Surya Jayasingh P

Assistant Professor

Dept. of CSE (AI)



Department of Computer Science and Engineering (Artificial Intelligence)

SREE NARAYANA MANGALAM INSTITUTE OF
MANAGEMENT AND TECHNOLOGY
MALIANKARA

October 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE)

SREE NARAYANA MANGALAM INSTITUTE OF MANAGEMENT &
TECHNOLOGY, MALIANKARA



CERTIFICATE

*This is to certify that the seminar report entitled **World Models for Embodied AI: Manipulation, Locomotion and Navigation** submitted by **Navaneeth K P (SNM23CA013)**, to the Department of Computer Science and Engineering (Artificial Intelligence), SNM Institute of Management and Technology, Maliankara, in partial fulfilment of the requirement for the award of B.Tech Degree in Computer Science and Engineering (Artificial Intelligence) is a bonafide record of the work carried out by him.*

Guide

Ms. Surya Jayasingh P
Asst. Professor
Dept. of CSE (AI)

Head of the Department

Ms. Lakshmi Anand
Asst. Professor
Dept. of CSE

Coordinator

Ms. Surya T S
Asst. Professor
Dept. of CSE

Declaration

I undersigned hereby declare that the seminar report **World Models for Embodied AI: Manipulation, Locomotion and Navigation**, submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Ms. Surya Jayasingh P**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Maliankara

Date: 30/09/2026

Navaneeth K P

SNM23CA013

Acknowledgement

Firstly, I thank God, the Almighty, with whose blessings, this seminar has been successfully completed.

I wish to give deep sense of acknowledgement to our Principal **Dr. Atmaram P**, for his constant encouragement and valuable advice throughout the course.

I am extremely grateful to **Ms. Lakshmi Anand**, Head of the Department of Computer Science and Engineering, for the valuable guidance through this humble endeavor. The highly enterprising attitude, patience for listening to my doubts, timely suggestions and guidance made my seminar a so-called reality in stipulated time. I would also like to thank our seminar coordinator **Ms. Surya T S**, Assistant Professor, Department of Computer Science and Engineering and my seminar guide **Ms. Surya Jayasingh P**, Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence) for their gracious of encouragement and the valuable assistance.

I express my sincere gratitude to all the faculty members of the department of Computer Science and Engineering (Artificial Intelligence) for their co-operation and support to the seminar.

I also express my sincere gratitude to all my friends and my parents for their co-operation and constant inspiration.

Navaneeth K P

Abstract

Embodied artificial intelligence requires agents that can perceive, predict and act in the physical world rather than merely map observations to actions. World models—internal representations that simulate how an environment evolves—have emerged as a unifying mechanism for this capability, yet their architectures diverge sharply across embodied domains. This seminar presents a comparative study of four recent IEEE works that build world models for distinct embodied tasks, spanning dexterous manipulation, legged locomotion and object goal navigation.

DexSim2Real², the base paper, constructs an explicit world model by actively interacting with an unseen articulated object, reconstructing a digital twin of it in a physics simulator through 3D AIGC and foundation vision models, and planning long-horizon trajectories using sampling-based Model Predictive Control (MPC), with Eigengrasp employed to reduce the high-dimensional action space of dexterous hands. Gaussian Action Field (GAF) adopts an implicit-geometric route, extending 3D Gaussian Splatting with learnable motion attributes to form a 4D spatiotemporal field from which future scene states and end-effector actions are directly inferred and then refined by an action-vision-aligned denoising module. World Model-based Perception (WMP) applies a latent Recurrent State-Space Model (RSSM) to visual legged locomotion, replacing the conventional teacher–student privileged-learning pipeline with a single-stage framework whose recurrent state compresses depth and proprioceptive history into a control-ready representation that transfers from simulation to hardware across gaps, climbs, tilts and crawls. WMNav pushes the abstraction furthest by treating a Vision-Language Model (VLM) itself as the world model for zero-shot object goal navigation, predicting the likelihood of a target’s presence into an online Curiosity Value Map and decomposing the goal into subtasks to suppress hallucination.

Comparing these systems along the axes of representation (explicit physics model, 4D Gaussian scene field, latent recurrent state, semantic-predictive reasoning), supervision cost, generalization to unseen objects and environments, and real-world deployability reveals a consistent trade-off between physical fidelity and semantic breadth. The study concludes that hybrid world models, combining geometric precision with commonsense reasoning, represent the most promising direction for the next generation of embodied agents.

Keywords: Embodied AI; World Models; Model-Based Reinforcement Learning (MBRL); Dexterous Manipulation; Sim2Real Transfer; 3D Gaussian Splatting (3DGS); Legged Locomotion; Object Goal Navigation; Vision-Language Models (VLM); Model Predictive Control (MPC).

Contents

Acknowledgement	I
Abstract	II
List of Figures	VII
List of Tables	VIII
List of Abbreviations	IX
1 Introduction	1
1.1 General Background	4
1.2 Evolution of World Models	5
1.3 Motivation	7
1.4 Objectives	7
1.5 Scope	8
1.6 Advantages	8
1.7 Organization of the Report	9
2 Theoretical Foundations of World Models	10
2.1 Embodied Decision Making as a POMDP	10
2.2 Model-Free versus Model-Based Learning	11
2.3 Formal Definition of a World Model	12
2.4 Latent World Models: The Recurrent State-Space Model	13
2.5 Planning with World Models: Sampling-Based MPC	14
2.6 Explicit Scene Representations: 3D Gaussian Splatting	16
2.7 Diffusion Models for Action Generation	17
2.8 Foundation Models as World Knowledge	18
2.9 Sim-to-Real Transfer	18
2.10 Evaluation Metrics	19
2.11 Taxonomy of World Models in Embodied AI	20
2.12 Chapter Summary	21
3 Literature Survey	22
3.1 DexSim2Real ² : Building Explicit World Model for Precise Articulated Ob- ject Dexterous Manipulation	22
3.1.1 Problem Statement and Motivation	23
3.1.2 Framework Overview	23

3.1.3	Interactive Perception	24
3.1.4	Explicit World Model Construction	25
3.1.5	Sampling-Based MPC with Eigengrasp	28
3.1.6	Experimental Setup	30
3.1.7	Results and Analysis	31
3.1.8	Discussion	34
3.2	GAF: Gaussian Action Field as a 4D Representation for Dynamic World Modeling in Robotic Manipulation	35
3.2.1	Problem Statement and Motivation	35
3.2.2	The Gaussian Action Field Representation	35
3.2.3	Architecture: Dynamic Gaussian Reconstruction	36
3.2.4	Init Action Computation	37
3.2.5	Manipulation with GAF: Action-Vision-Aligned Denoising	38
3.2.6	Experimental Setup and Implementation	38
3.2.7	Results and Analysis	39
3.2.8	Discussion	41
3.3	World Model-based Perception for Visual Legged Locomotion (WMP)	42
3.3.1	Problem Statement and Motivation	42
3.3.2	Problem Formulation	43
3.3.3	World Model Learning	44
3.3.4	Policy Learning	45
3.3.5	Training Environment and Rewards	45
3.3.6	Simulation Results	46
3.3.7	Empirical Analysis of the World Model	47
3.3.8	Real-World Evaluation	48
3.3.9	Discussion	49
3.4	WMNav: Integrating Vision-Language Models into World Models for Object Goal Navigation	49
3.4.1	Problem Statement and Motivation	50
3.4.2	Task Definition and Framework Overview	50
3.4.3	World Model: Prediction and Curiosity Value Map	51
3.4.4	Subtask Decomposition	52
3.4.5	Two-Stage Action Proposer	53
3.4.6	Experimental Setup	54
3.4.7	Results and Analysis	55
3.4.8	Discussion	56
4	Discussion & Comparison	57
4.1	Analytical Focus	57
4.2	Comparison of Methodologies	58
4.3	Representation, Learning Signal, and Control Strategy	59

4.4	Model Performance and Evaluation	60
4.5	Data and Environment Utilization	61
4.6	Scalability and Deployment Potential	62
4.7	Interpretability and Trustworthiness	62
4.8	A Design-Space View	63
4.9	Innovation and Research Contributions	64
4.10	Practical Relevance and Real-World Application	64
4.11	Overall Comparative Insights	65
5	Applications	66
5.1	Household and Assistive Robotics	66
5.2	Industrial and Warehouse Manipulation	66
5.3	Inspection, Search-and-Rescue, and Field Robotics	67
5.4	Domestic Service and Semantic Navigation	67
5.5	Healthcare and Laboratory Automation	67
5.6	Simulation, Testing, and Education	68
6	Challenges and Future Directions	69
6.1	Challenges Identified in the Reviewed Works	69
6.1.1	DexSim2Real ² : Explicit World Model for Dexterous Manipulation	69
6.1.2	GAF: 4D Gaussian World Modeling	69
6.1.3	WMP: World Model-based Perception for Locomotion	70
6.1.4	WMNav: VLM World Models for Navigation	70
6.2	Cross-Cutting Challenges	70
6.3	Future Research Directions	72
6.4	Ethical and Societal Considerations	73
6.5	Summary	73
7	Conclusion	74
	References	76

List of Figures

2.3.1	The generic perception–prediction–action loop of a world-model-based agent. Candidate actions are evaluated inside the world model (dashed arrows) before a single action is sent to the real environment.	13
2.4.1	The Recurrent State-Space Model unrolled over time. Rectangles are deterministic recurrent states, green circles stochastic latent states, and grey circles observations. In WMP the recurrent state is updated only every k control steps using the accumulated action sequence.	14
2.11.1	Taxonomy of world-model approaches in embodied AI	20
3.1.1	Overview of the DexSim2Real ² framework [1]	24
3.1.2	(a) Generating a real robot manipulation trajectory from 2D affordances; (b) computing the 3D post-contact vector as the intersection of two projection planes [1]	26
3.1.3	Explicit world model construction pipeline: 3D AIGC mesh generation, pose and scale estimation, movable part segmentation, and kinematic structure estimation [1]	26
3.1.4	Accumulated variance ratio versus eigengrasp dimension for the Allegro Hand and the XHand [1]	29
3.1.5	Quantitative results of DexSim2Real ² plotted from the values reported in [1] (lower is better)	33
3.1.6	Generalization of the explicit world model: (a) reconstructed twins of a cabinet with two revolute joints and a refrigerator with two revolute joints (pink) and one prismatic joint (blue); (b) tool use planned entirely in the world model [1]	34
3.2.1	Comparison of the V-A, V-3D-A, and the proposed V-4D-A (GAF) paradigms [2]	36
3.2.2	GAF reconstruction: a ViT with cross-view attention and three heads predicts Gaussian centres, appearance parameters, and motion [2]	37
3.2.3	GAF manipulation pipeline with action-vision-aligned denoising [2] . . .	39
3.2.4	Average RL Bench success rate of GAF (orange) compared with baselines and its own ablation (blue), plotted from [2]	40
3.2.5	Real-world GAF results: renderings from current and predicted future Gaussians on four tasks [2]	41

3.3.1	Scandots (left) and corresponding depth images (right) for Tilt and Crawl terrains: scandots cannot distinguish precise boundaries or represent off-ground obstacles [3]	43
3.3.2	The WMP framework: the RSSM world model runs every k steps and its recurrent state feeds the policy [3]	44
3.3.3	Simulated returns of WMP, the privileged-learning Student, and the Blind ablation on six terrains, plotted from [3]. The Student is not applicable to Tilt and Crawl (no bar).	47
3.3.4	Empirical study of the WMP world model [3]	47
3.3.5	Real-world depth images and long-horizon predictions of the WMP world model on Gap, Climb, Tilt, and Crawl [3]	48
3.3.6	Real-world success rates of WMP, Student, Blind, and WMP w/o Prop over ten trials per difficulty level [3]	49
3.4.1	World model navigation with a VLM: the model estimates the goal’s likelihood in each panoramic direction, plans an intermediate subtask, and selects an action [4]	50
3.4.2	The WMNav framework with VLM-based world model and navigation policy [4]	51
3.4.3	(a) Predicting curiosity values for each panoramic direction; (b) projection of navigable areas between egocentric and top-down views; (c) fusing predictions into the Curiosity Value Map [4]	52
3.4.4	Reasoning the action: the exploration stage filters sampled actions for ActionVLM, and the goal-approaching stage densely samples positions for GoalVLM [4]	54
3.4.5	WMNav performance on HM3D v0.1 with different VLMs, plotted from [4]	56
4.8.1	Qualitative placement of the reviewed world models in a fidelity–generality design space. Positions are a qualitative synthesis of the comparison in this chapter, not measured quantities.	63

List of Tables

1.1	Milestones in the evolution of world models	6
2.1	Comparison of model-free and model-based approaches to embodied control	12
2.2	Families of world models and their characteristics	21
3.1	Sampling-based MPC settings used in DexSim2Real ² [1]	30
3.2	Accuracy of real articulated object manipulation with the suction gripper [1]	31
3.3	Accuracy of real manipulation with the two-finger gripper and the dexter- ous hand [1]	32
3.4	Explicit world model construction accuracy [1]	33
3.5	Implementation details of GAF [2]	39
3.6	Success rates (%) on nine RL Bench tasks [2]	40
3.7	Average novel-view synthesis quality for current and future scenes (three RL Bench tasks) [2]	41
3.8	Simulation comparison of WMP (return $\uparrow$ / velocity tracking error $\downarrow$), mean over 100 trajectories [3]	46
3.9	Agent configuration used in WMNav experiments [4]	54
3.10	Object navigation results on HM3D v0.1 and MP3D. TF: training-free; ZS: zero-shot [4]	55
3.11	Ablation of modules and memory strategies on HM3D v0.2. SD: subtask decomposition; TAP: two-stage action proposer; CVM: Curiosity Value Map [4]	56
4.1	Comparison of world-model approaches for embodied AI	58
4.2	World-model representation, construction, and use in the four reviewed works	60
4.3	Experimental platforms and evaluation protocols	61
5.1	Application domains and the most suitable world-model families	68

List of Abbreviations

3DGS	3D Gaussian Splatting
AIGC	Artificial Intelligence Generated Content
AMP	Adversarial Motion Priors
CEM	Cross-Entropy Method
CMA-ES	Covariance Matrix Adaptation Evolution Strategy
CNN	Convolutional Neural Network
DDIM	Denoising Diffusion Implicit Model
DDPM	Denoising Diffusion Probabilistic Model
DOF	Degrees of Freedom
DPT	Dense Prediction Transformer
ELBO	Evidence Lower Bound
GAF	Gaussian Action Field
GRU	Gated Recurrent Unit
HM3D	Habitat-Matterport 3D Dataset
ICP	Iterative Closest Point
iCEM	Improved Cross-Entropy Method
IL	Imitation Learning
IoU	Intersection over Union
KL	Kullback–Leibler (divergence)
LLM	Large Language Model
LPIPS	Learned Perceptual Image Patch Similarity
MBRL	Model-Based Reinforcement Learning
MLP	Multi-Layer Perceptron
MP3D	Matterport3D Dataset
MPC	Model Predictive Control
MPPI	Model Predictive Path Integral Control
NeRF	Neural Radiance Field
PCA	Principal Component Analysis
PD	Proportional–Derivative (controller)
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization

PSNR	Peak Signal-to-Noise Ratio
RDE	Relative Depth Error
RGBD	Red Green Blue plus Depth
RL	Reinforcement Learning
RSSM	Recurrent State-Space Model
SE(3)	Special Euclidean Group in three dimensions
SPL	Success weighted by Path Length
SR	Success Rate
SSIM	Structural Similarity Index Measure
t-SNE	t-distributed Stochastic Neighbour Embedding
URDF	Unified Robot Description Format
V-4D-A	Vision-to-4D-to-Action
VHACD	Volumetric Hierarchical Approximate Convex Decomposition
ViT	Vision Transformer
VLM	Vision-Language Model
VRB	Vision-Robotics Bridge
WMP	World Model-based Perception
ZSON	Zero-Shot Object Navigation

Chapter 1

Introduction

Embodied Artificial Intelligence (AI) concerns intelligent agents—most prominently robots—that must perceive their surroundings, reason about the physical world, and execute actions to accomplish goals. Unlike disembodied AI systems that operate purely on digital data, embodied agents are embedded in dynamic, partially observable, and physically constrained environments. A robot that manipulates a cabinet, a quadruped that climbs over an obstacle, and a domestic robot that searches for a bed in an unfamiliar house all face the same fundamental difficulty: a single observation rarely reveals everything the agent needs to know, and every action has physical consequences that unfold over time. Purely reactive policies, which map current perception directly to actions, often struggle in such settings because they cannot anticipate the outcome of an action before committing to it.

Humans and animals solve this problem with remarkable elegance. Cognitive science suggests that biological agents build a **mental model** of the world—an internal representation that captures how the environment behaves and how it will respond to actions. Using motor imagery and mental simulation, humans can “imagine” the consequences of a movement before executing it, and they actively interact with objects to gather the information required to refine this model, a process known as **interactive perception**. Translating this natural ability to machines has given rise to the concept of the **world model** [5]: a computational representation of environment dynamics that allows an agent to predict future states, simulate candidate actions internally, and plan behavior with foresight rather than reacting blindly.

World models have a long history in optimal control and Model-Based Reinforcement Learning (MBRL), where a learned transition model of the environment is used to improve sample efficiency and enable planning. Modern instantiations span a wide spectrum. At one end are **explicit physics-based world models**, in which the environment is reconstructed as an interactable digital twin inside a physics simulator; these models embed strong physical priors and generalize naturally to unseen actions. At the other end are **learned latent world models**, such as the Recurrent State-Space Model (RSSM) popularized by the Dreamer family, which compress high-dimensional sensory streams into compact latent states that support future prediction. Between these extremes lie structured scene-level representations, such as dynamic 3D Gaussian fields that attach motion attributes to explicit geometry, and, most recently, **foundation-model-driven world models**, in which Large Language Models (LLMs) and Vision-Language Models (VLMs) serve as commonsense predictors of how a scene is likely to unfold.

This seminar examines how these different families of world models are applied across the three core competencies of embodied AI. In the domain of **manipulation**, the base paper DexSim2Real² [1] constructs an explicit world model of unseen articulated objects, such as drawers, cabinets, laptops, and lamps, through one-step interactive perception followed by 3D AI-Generated Content (AIGC)-based digital twin reconstruction; sampling-based Model Predictive Control (MPC) then plans precise, goal-conditioned trajectories for suction grippers, two-finger grippers, and multi-finger dexterous hands without requiring demonstrations or reinforcement learning. Complementing this, the Gaussian Action Field (GAF) [2] adopts a Vision-to-4D-to-Action (V-4D-A) paradigm: it extends 3D Gaussian Splatting (3DGS) with learnable motion attributes so that a single feed-forward network reconstructs the current scene, predicts the future scene, and extracts an initial action from the predicted Gaussian motion, which is subsequently refined by an action-vision-aligned diffusion model.

In the domain of **locomotion**, World Model-based Perception (WMP) [3] addresses visual legged locomotion over challenging terrains such as gaps, climbs, tilts, and crawl spaces. Instead of the conventional privileged teacher–student framework, WMP trains an

RSSM world model entirely in simulation to predict future depth images and proprioception, and feeds the recurrent latent state of this model to a Proximal Policy Optimization (PPO) policy. Although trained purely on simulated data, the world model produces accurate predictions of real-world trajectories, enabling a Unitree A1 quadruped to traverse a gap of 85 cm (about $2.1\times$ its body length) and a climb of 55 cm (about $2.2\times$ its height). In the domain of **navigation**, WMNav [4] integrates VLMs into a modular world model for Zero-Shot Object Navigation (ZSON): a PredictVLM estimates the likelihood that the target object lies in each direction, an online Curiosity Value Map accumulates these predictions as spatial memory, and subtask decomposition with a two-stage action proposer converts the predictions into efficient navigation actions, achieving state-of-the-art success rates on the HM3D and MP3D benchmarks.

Viewed together, the four systems reveal a common architectural philosophy—predict before acting—realized through very different design choices. DexSim2Real² places its knowledge in an explicit simulator and searches for trajectories; GAF embeds dynamics directly into an explicit scene representation and reads actions out of predicted motion; WMP compresses history into a latent state that a learned policy consumes; and WMNav delegates prediction to the commonsense knowledge stored in pre-trained VLMs. These choices induce distinct trade-offs in data requirements, computational cost, precision, interpretability, and generality, which this report analyzes in depth.

Despite their successes, world-model-based embodied AI still faces open **challenges**. Explicit reconstruction pipelines depend on the fidelity of 3D generative models and can be misled by novel geometry; latent models trained in simulation inherit the sim-to-real gap of their training distribution; VLM-based predictors are susceptible to hallucination; and all approaches must balance model accuracy against the latency constraints of real-time control. Emerging directions such as training world models on mixtures of simulated and real data, incorporating tactile and other sensing modalities, scene-level digital twins for mobile manipulation, and hallucination-resistant prompting strategies aim to address these limitations and are discussed throughout this report.

1.1 General Background

Approaches to embodied decision-making can be broadly categorized into **model-free**, **model-based**, and **hybrid** paradigms. Model-free methods, including most Reinforcement Learning (RL) and Imitation Learning (IL) policies, learn a direct mapping from observations to actions; they are simple to deploy but data-hungry and brittle when the state distribution is complex, as with articulated objects. Model-based methods first acquire a model of the environment—explicit or learned—and then use it for planning or policy optimization, gaining sample efficiency and foresight at the cost of model construction. Hybrid systems combine both: a generalizable model-construction module paired with task-specific planning (as in DexSim2Real²), or a learned world model whose latent state feeds a model-free policy (as in WMP). Architecturally, world models exploit components such as Convolutional Neural Networks (CNNs) for visual encoding, Gated Recurrent Units (GRUs) for temporal dynamics, Vision Transformers (ViTs) for geometry-aware feature extraction, and foundation VLMs for semantic prediction, allowing embodied agents to perceive, remember, and predict in real time.

A second axis along which embodied systems differ is the **source of their training signal**. Early robot learning relied on hand-engineered controllers and carefully tuned state machines. The deep learning era replaced these with policies trained either from human demonstrations (imitation) or from trial-and-error interaction with a reward signal (reinforcement). Both strategies require large quantities of data: dexterous in-hand manipulation and agile locomotion policies are typically trained on billions of simulated interaction steps, while imitation-based manipulation systems need hundreds of demonstrations per task. Physics simulators such as SAPIEN and Isaac Gym alleviate this burden by providing massively parallel, safe, and inexpensive experience, but they introduce the **sim-to-real gap**: discrepancies in dynamics, sensing, and appearance between the simulated and the physical world. World models sit at the heart of this landscape because they can be learned from simulation, from real data, or from internet-scale pre-training, and because they allow the agent to reuse a single predictive model for many downstream tasks.

A third axis is the **modality of perception**. Manipulation systems typically rely on RGB or RGB-D cameras observing a tabletop; legged robots combine proprioception (joint angles, velocities, body orientation) with egocentric depth images; navigation agents use panoramic RGB-D views and pose estimates. Each modality imposes its own requirements on the world model. Manipulation needs fine geometric accuracy at the contact level, locomotion needs fast and robust estimates of terrain geometry under motion blur and latency, and navigation needs semantic understanding of room layouts and object–room relationships.

1.2 Evolution of World Models

The idea that an agent should learn and exploit a model of its environment predates deep learning by decades. Sutton’s **Dyna** architecture [6] (1991) integrated learning, planning, and reacting by using a learned model to generate simulated experience that supplements real interaction. Model Predictive Control, developed in the process-control industry, formalized the practice of repeatedly optimizing a finite-horizon plan through a dynamics model and executing only its first action. These classical approaches, however, relied on low-dimensional, hand-specified state spaces.

The modern resurgence of world models began when deep networks made it possible to learn dynamics directly from pixels. Ha and Schmidhuber’s **World Models** (2018) combined a variational autoencoder, a recurrent mixture-density network, and a compact controller, showing that an agent could learn to act inside its own “dream”. PETS and MBPO demonstrated that probabilistic ensembles of learned dynamics could match model-free performance with an order of magnitude fewer samples. **PlaNet** [7] introduced the Recurrent State-Space Model (RSSM), which separates deterministic and stochastic latent components, and the **Dreamer** series [8] learned behaviors entirely by imagination in this latent space, eventually mastering diverse domains with a single configuration. **DayDreamer** then showed that such models could train real robots, including a quadruped that learned to walk in about an hour, directly in the physical world.

In parallel, the robotics community developed **explicit** world models in the form of digital twins: reconstructions of objects or scenes that can be loaded into physics engines. Ditto built digital twins of single-joint articulated objects from two point-cloud frames, and Sim2Real², the predecessor of the base paper, used such a twin as a physics-based “mental model” for precise manipulation. The arrival of Neural Radiance Fields and 3D Gaussian Splatting provided photorealistic, differentiable scene representations that could be extended in time, leading to Gaussian world models such as ManiGaussian and GAF [2]. Most recently, large video-prediction models and foundation models have been proposed as general-purpose world simulators, and LLMs/VLMs have been used as abstract world models for web agents and embodied navigation [4]. Table 1.1 summarizes this progression.

Table 1.1: Milestones in the evolution of world models

Year	Work	Key contribution
1991	Dyna	Learned model generates simulated experience for planning and learning.
2018	World Models	VAE + recurrent dynamics; policies trained inside a learned “dream”.
2018–19	PETS / MBPO	Probabilistic dynamics ensembles; sample-efficient continuous control.
2019	PlaNet	Recurrent State-Space Model; latent planning from pixels.
2019–23	Dreamer V1–V3	Behaviour learning by latent imagination across diverse domains.
2022	Ditto	Digital twins of articulated objects from interaction.
2023	DayDreamer	World models trained directly on physical robots.
2023	TD-MPC2	Scalable latent world models combined with MPC.
2023	3D Gaussian Splatting	Real-time, differentiable explicit scene representation.
2024	WMP [3]	RSSM world model as perception for visual legged locomotion.
2025	DexSim2Real ² [1]	3D AIGC digital twins + MPC for dexterous articulated manipulation.
2025	WMNav [4]	VLMs as world models for zero-shot object navigation.
2026	GAF [2]	4D Gaussian Action Field for dynamic world modelling in manipulation.

1.3 Motivation

The motivation for this seminar arises from a practical observation: the most capable embodied systems published in the last two years increasingly contain an explicit *prediction* component, yet the form of that component varies enormously from one application to another. A student or practitioner entering the field is confronted with digital twins, latent state-space models, Gaussian fields, and prompted foundation models, all described as “world models”. Understanding what these approaches share, how they differ, and when each is appropriate is essential for designing future robotic systems.

A second motivation is the growing importance of **data efficiency**. Collecting real robot data is slow, expensive, and sometimes dangerous. World models offer a way to amortize experience: a single model can be reused for many goals (as in DexSim2Real², where one digital twin supports opening and closing to arbitrary joint angles), can be trained in simulation and transferred (as in WMP), or can be borrowed from internet-scale pre-training (as in WMNav). Examining these strategies side by side clarifies the trade-offs between collecting data, building models, and exploiting prior knowledge.

Finally, world models are closely linked to **safety and interpretability**. A robot that can simulate the consequences of its actions can reject unsafe plans before executing them, and a model that can be rendered or queried offers a window into the agent’s beliefs. As robots move from laboratories into homes, warehouses, and outdoor environments, these properties become as important as raw task performance.

1.4 Objectives

The principal objectives of this seminar are:

- To introduce the concept of world models and the theoretical foundations—POMDPs, latent state-space models, sampling-based MPC, Gaussian splatting, diffusion models, and foundation models—on which modern embodied world models are built.
- To study in depth the base paper, DexSim2Real², including its interactive perception

module, its 3D AIGC-based explicit world model construction pipeline, and its eigengrasp-accelerated sampling-based MPC.

- To review three complementary works—GAF for manipulation, WMP for legged locomotion, and WMNav for object goal navigation—and to extract their methodology, experimental protocol, and key results.
- To compare the four approaches along representation, data requirements, computational cost, interpretability, generalization, and real-world deployability.
- To identify the open challenges and promising future research directions for world-model-driven embodied intelligence.

1.5 Scope

The scope of this seminar is to explore and evaluate world-model-driven advancements across the principal domains of embodied AI. It draws from four core research studies focusing on: paper [1] the construction of explicit digital-twin world models of articulated objects through interactive perception and 3D AIGC for precise dexterous manipulation; paper [2] 4D spatiotemporal world modeling with Gaussian Action Fields that unify scene reconstruction, future prediction, and action generation; paper [3] latent RSSM world models as perceptual backbones for vision-based legged locomotion with sim-to-real transfer; and paper [4] VLM-powered predictive world models with curiosity-based spatial memory for zero-shot object goal navigation. The seminar further discusses the evaluation methodologies of these systems (manipulation error, success rate, reconstruction quality, traversal difficulty, SR/SPL navigation metrics), their experimental platforms, and the practical challenges of real-world deployment. Large-scale video-generation world models for autonomous driving and general-purpose foundation world simulators are mentioned for context but are outside the detailed scope of this report.

1.6 Advantages

The integration of world models into embodied AI offers numerous advantages that transform reactive robotic systems into predictive, anticipatory agents:

- **Foresight and Planning:** Agents simulate the consequences of candidate actions

internally before physical execution, enabling long-horizon planning.

- **Data Efficiency:** Explicit physics priors (DexSim2Real²) and learned dynamics (WMP) drastically reduce the number of real-world interactions required.
- **Demonstration-Free Operation:** Sampling-based MPC over a constructed world model eliminates the need for human demonstrations or reward engineering per task.
- **Precision:** Digital-twin-based planning achieves goal-conditioned manipulation with average errors as low as 7° on real articulated objects.
- **Generalization to Unseen Actions:** An explicit model supports novel strategies never seen during data collection, such as tool-based drawer opening.
- **Robust Sim-to-Real Transfer:** World models trained in simulation predict real-world trajectories accurately, shrinking the sim-to-real gap in locomotion.
- **Reduced Risky Interactions:** Predicting outcomes before acting reduces collisions and costly exploratory failures in navigation.
- **Zero-Shot Semantic Reasoning:** VLM-based world models exploit internet-scale commonsense to navigate to unseen object categories without task-specific training.
- **Unified Perception–Prediction–Action:** Representations such as GAF couple reconstruction, prediction, and action inference in one framework.
- **Cross-Domain Applicability:** The same predictive philosophy applies to manipulation, locomotion, and navigation, indicating a general paradigm for robot control.

1.7 Organization of the Report

The remainder of this report is organized as follows. **Chapter 2** develops the theoretical foundations of world models. **Chapter 3** presents the literature survey, reviewing the base paper DexSim2Real² and the three complementary papers in detail. **Chapter 4** compares the four approaches, **Chapter 5** surveys practical applications, **Chapter 6** discusses open challenges and future research directions, and **Chapter 7** concludes the report.

Chapter 2

Theoretical Foundations of World Models

Before examining the four reviewed systems, this chapter introduces the mathematical and conceptual building blocks on which they rest. Each concept is presented in the form in which it is later used: the POMDP formulation underlies WMP; latent state-space models and variational training underlie the RSSM; sampling-based model predictive control underlies DexSim2Real²; Gaussian splatting and diffusion models underlie GAF; and foundation models underlie WMNav.

2.1 Embodied Decision Making as a POMDP

An embodied agent interacting with its environment is most naturally described as a **Partially Observable Markov Decision Process** (POMDP), defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, r, \gamma)$, where $\mathcal{S}$, $\mathcal{O}$, and $\mathcal{A}$ are the state, observation, and action spaces, $\mathcal{T}(s_{t+1} | s_t, a_t)$ is the transition density, $r(s_t, a_t)$ is the reward function, and $\gamma \in (0, 1)$ is a discount factor [3]. At every time step the agent receives only a partial observation $o_t \in \mathcal{O}$ produced by its sensors, not the full state s_t . The goal of reinforcement learning is to find the policy that maximizes the expected discounted return:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{s_{t+1} \sim \mathcal{T}(\cdot | s_t, a_t), a_t \sim \pi(\cdot | o_{\leq t})} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]. \quad (2.1)$$

Partial observability is the central difficulty of embodied AI. A forward-facing camera on a quadruped cannot see the terrain directly beneath its feet; a single RGB-D view of a cabinet cannot reveal whether its door rotates about a hinge or slides on a rail; and a navigation agent cannot see into the next room. Because the optimal action depends on

quantities that are not directly observed, the agent must maintain some internal summary of its history—a *belief state* $b_t = p(s_t \mid o_{\leq t}, a_{< t})$. Computing exact beliefs is intractable for high-dimensional sensory inputs, which is precisely why learned or constructed world models are needed: they provide a tractable approximation of the belief together with the ability to predict how it will evolve under candidate actions.

Policies for continuous control are frequently optimized with **Proximal Policy Optimization** (PPO) [9], an actor–critic algorithm that constrains each policy update to a trust region through a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}, \quad (2.2)$$

where $\hat{A}_t$ is an advantage estimate obtained from a learned value function (the critic). PPO is simple, stable, and parallelizes well across thousands of simulated robots, which makes it the de facto standard for learning locomotion in simulation and the algorithm used by WMP.

2.2 Model-Free versus Model-Based Learning

Model-free methods learn a policy or value function directly from experience without ever representing how the world works. They are general and asymptotically strong, but they typically require very large amounts of interaction, and the resulting policy is tied to the task and reward it was trained on. **Model-based** methods first learn (or construct) a model $\hat{\mathcal{T}}$ of the environment and then derive behavior from it, either by *planning*—searching over action sequences inside the model at decision time—or by *policy learning in imagination*—training a policy on trajectories generated by the model. The model can be reused across tasks, supports counterfactual reasoning, and usually yields much better sample efficiency. Its weakness is *model bias*: errors in the model can be exploited by the planner or policy and lead to poor real-world behavior. Table 2.1 contrasts the two families.

Table 2.1: Comparison of model-free and model-based approaches to embodied control

Aspect	Model-free (RL / IL policy)	Model-based (world model)
Core idea	Map observations directly to actions	Predict future states, then plan or learn in imagination
Sample efficiency	Low; millions of interactions or hundreds of demonstrations	High; the model amortizes experience across tasks
Reuse across goals	Poor; policy is task-specific	Good; the same model supports many goals and rewards
Computation at run time	Very low (one network forward pass)	Higher when planning (many model rollouts per step)
Main failure mode	Distribution shift, poor generalization	Model bias and compounding prediction error
Interpretability	Low	Higher; predictions can be rendered and inspected
Examples in this report	Student policy baseline, Diffusion Policy	DexSim2Real ² , GAF, WMP, WMNav

2.3 Formal Definition of a World Model

In its most general form, a world model is a collection of learned or constructed functions that allow an agent to answer the question “*what will happen if I do this?*”. Following the Dreamer-style decomposition, a world model contains four components:

$$\text{Representation model: } z_t \sim q_\phi(z_t \mid z_{t-1}, a_{t-1}, o_t), \quad (2.3)$$

$$\text{Transition model: } \hat{z}_{t+1} \sim p_\phi(\hat{z}_{t+1} \mid z_t, a_t), \quad (2.4)$$

$$\text{Observation model: } \hat{o}_t \sim p_\phi(\hat{o}_t \mid z_t), \quad (2.5)$$

$$\text{Reward model: } \hat{r}_t \sim p_\phi(\hat{r}_t \mid z_t). \quad (2.6)$$

The representation model compresses the raw observation into a state z_t ; the transition model predicts how that state evolves under an action; the observation model reconstructs or renders what the agent would see; and the reward model predicts task progress. Given an initial state and a candidate action sequence $a_{t:t+H}$, the agent can *imagine* a trajectory $\hat{z}_{t+1:t+H+1}$ and evaluate its predicted return $\sum_\tau \gamma^{\tau-t} \hat{r}_\tau$ without touching the real environment. Figure 2.3.1 illustrates this perception–prediction–action loop.

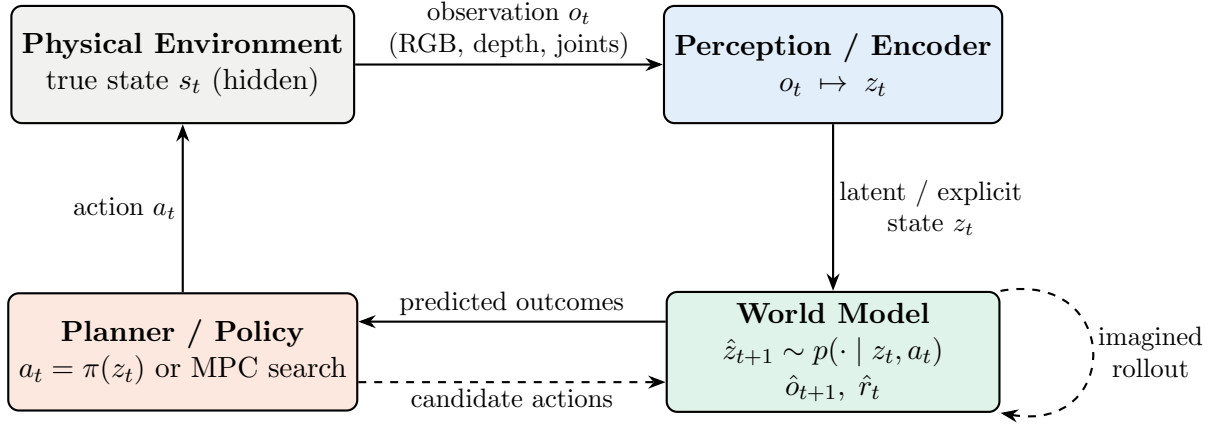


Figure 2.3.1: The generic perception–prediction–action loop of a world-model-based agent. Candidate actions are evaluated inside the world model (dashed arrows) before a single action is sent to the real environment.

The four reviewed works instantiate the components of this loop very differently. In DexSim2Real², the “state” is an explicit URDF model loaded into the SAPIEN simulator and the transition model is the physics engine itself. In GAF, the state is a set of 3D Gaussians and the transition model is a learned per-Gaussian displacement field. In WMP, the state is the recurrent latent h_t of an RSSM and the transition model is a GRU. In WMNav, the “state” is a Curiosity Value Map and the transition model is a VLM prompted to imagine what lies in each direction. This diversity is the central theme of the report.

2.4 Latent World Models: The Recurrent State-Space Model

The Recurrent State-Space Model (RSSM), introduced in PlaNet and refined in the Dreamer series, is the most widely used latent world model. Its key insight is to split the latent state into a *deterministic* recurrent component h_t , which carries information reliably over long horizons, and a *stochastic* component z_t , which captures uncertainty and multi-modality. The model consists of

$$\text{Recurrent model: } h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \quad (2.7)$$

$$\text{Encoder (posterior): } z_t \sim q_\phi(z_t | h_t, o_t), \quad (2.8)$$

$$\text{Dynamics predictor (prior): } \hat{z}_t \sim p_\phi(\hat{z}_t | h_t), \quad (2.9)$$

$$\text{Decoder: } \hat{o}_t \sim p_\phi(\hat{o}_t | h_t, z_t). \quad (2.10)$$

The recurrent model is typically a GRU. The posterior incorporates the current observation, while the prior must predict the stochastic state from history alone; at deployment, rolling the prior forward allows open-loop prediction of future observations.

The RSSM is trained by maximizing a variational evidence lower bound (ELBO), which in practice becomes the minimization of a reconstruction loss plus a Kullback–Leibler (KL) regularizer over sequences of length L :

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi} \left[\sum_{t=1}^L -\ln p_\phi(o_t | h_t, z_t) + \beta \text{KL}(q_\phi(z_t | h_t, o_t) \parallel p_\phi(z_t | h_t)) \right]. \quad (2.11)$$

The reconstruction term forces the latent state to retain information about the observation, and the KL term makes the prior predictive of the posterior, so that the model can “imagine” without observations. Figure 2.4.1 shows the RSSM unrolled in time with the reduced update frequency used by WMP.

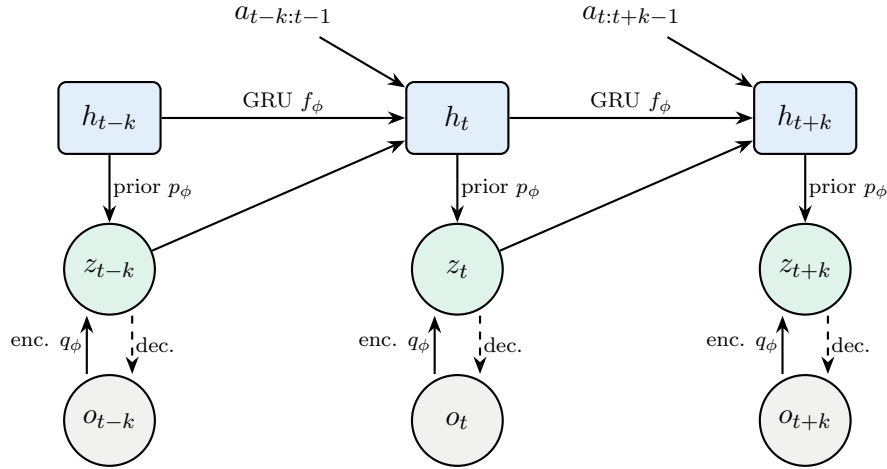


Figure 2.4.1: The Recurrent State-Space Model unrolled over time. Rectangles are deterministic recurrent states, green circles stochastic latent states, and grey circles observations. In WMP the recurrent state is updated only every k control steps using the accumulated action sequence.

2.5 Planning with World Models: Sampling-Based MPC

Model Predictive Control (MPC) uses a world model to repeatedly solve a finite-horizon optimization problem and executes only the first action of the resulting plan

before re-planning. At time t with planning horizon h , MPC solves

$$a_{t:t+h-1}^* = \arg \max_{a_{t:t+h-1}} \sum_{\tau=t}^{t+h-1} r(\hat{s}_\tau, a_\tau), \quad \hat{s}_{\tau+1} = f(\hat{s}_\tau, a_\tau), \quad \hat{s}_t = s_t, \quad (2.12)$$

where f is the world model. When the model is a physics simulator with complex contacts, as in DexSim2Real², gradients of the objective are unavailable or uninformative, so **zeroth-order** (sampling-based) optimizers are used. The best known are the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), the **Cross-Entropy Method** (CEM), and Model Predictive Path Integral control (MPPI).

CEM maintains a Gaussian distribution $\mathcal{N}(\mu, \text{diag}(\sigma^2))$ over action sequences. In each iteration it samples a population of N sequences, evaluates each by rolling it out in the model, selects the E highest-scoring sequences (the *elite set*), and refits the distribution to the elites:

$$\mu \leftarrow \frac{1}{E} \sum_{e \in \mathcal{E}} A^{(e)}, \quad \sigma^2 \leftarrow \frac{1}{E} \sum_{e \in \mathcal{E}} (A^{(e)} - \mu)^2. \quad (2.13)$$

MPPI instead updates the mean with an exponentially weighted average of *all* samples, $\mu \leftarrow \sum_n w_n A^{(n)}$ with $w_n \propto \exp(R^{(n)}/\lambda)$, where λ is a temperature. The **improved CEM** (iCEM) [10] used by DexSim2Real² adds several refinements that reduce the number of samples needed for real-time planning: temporally correlated (coloured-noise) sampling that produces smoother action sequences, keeping a fraction of elites between iterations and between time steps, shifting the previous solution to initialize the next step, and executing the best sampled action rather than the mean. Algorithm 1 summarizes sampling-based MPC.

The computational cost of sampling-based MPC grows with the dimension of the action space, because the number of samples needed to cover the space grows rapidly with dimension. This is why DexSim2Real² employs **eigengrasps**: Principal Component Analysis (PCA) is applied to a large dataset of hand postures, and the hand configuration is

Algorithm 1 Sampling-based MPC with the (improved) Cross-Entropy Method

Require: world model f , reward r , horizon h , population N , elite size E , iterations I , trajectory length T

- 1: Initialise mean $\mu_{0:h-1} \leftarrow 0$ and standard deviation $\sigma_{0:h-1}$
- 2: **for** $t = 0, \dots, T - 1$ **do**
- 3: **for** $i = 1, \dots, I$ **do**
- 4: Sample N action sequences $A^{(n)} \sim \mathcal{N}(\mu, \text{diag}(\sigma^2))$ $\triangleright$ coloured noise in iCEM
- 5: Roll out each $A^{(n)}$ in the world model from s_t ; compute $R^{(n)} = \sum_{\tau} r(\hat{s}_{\tau}, a_{\tau}^{(n)})$
- 6: Select the E sequences with the highest $R^{(n)}$ as elite set $\mathcal{E}$ $\triangleright$ reuse elites in iCEM
- 7: Refit μ, σ to $\mathcal{E}$ using Eq. (2.13)
- 8: **end for**
- 9: Apply the first action of the best sequence, observe s_{t+1}
- 10: Shift μ by one step to warm-start the next optimisation
- 11: **end for**
- 12: **return** executed trajectory $a_{0:T-1}$

expressed as a linear combination of the first m principal components,

$$q_h = \bar{q}_h + \sum_{i=1}^m \alpha_i e_i, \quad m \ll d_{\text{hand}}, \quad (2.14)$$

where e_i are the eigenvectors with the largest eigenvalues and α_i are the low-dimensional coefficients searched by the planner. Because human and robot grasps are highly correlated across joints, a handful of components captures most of the variance of natural hand postures.

2.6 Explicit Scene Representations: 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) [11] represents a scene as a set of anisotropic 3D Gaussians, each with a centre $\mu \in \mathbb{R}^3$, a covariance Σ , an opacity σ , and a colour c (often represented with spherical harmonics). The density of a Gaussian at point x is

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^{\top} \Sigma^{-1} (x - \mu)\right), \quad \Sigma = R S S^{\top} R^{\top}, \quad (2.15)$$

where the covariance is factorized into a rotation R (stored as a quaternion r) and a diagonal scale matrix S (stored as a vector s) to guarantee positive semi-definiteness. To render an image, each Gaussian is projected to the image plane with covariance $\Sigma' =$

$JW\Sigma W^\top J^\top$, where W is the viewing transformation and J the Jacobian of the projective transformation, and the pixel colour is obtained by front-to-back alpha blending of the N depth-sorted Gaussians that overlap the pixel:

$$C(p) = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad \alpha_i = \sigma_i e^{-\frac{1}{2}(p - \mu_i^{2d})^\top \Sigma_i'^{-1} (p - \mu_i^{2d})}. \quad (2.16)$$

Because this rasterization is differentiable, the Gaussian parameters can be optimized directly from multi-view images using photometric losses, without any 3D ground truth. Compared with Neural Radiance Fields, which query a neural network at many points along each camera ray, 3DGS renders in real time and provides an explicit point-based geometry that can be manipulated, segmented, and registered. These properties make it attractive as a world representation for robotics: GAF [2] extends each Gaussian with a motion vector $\Delta\mu$ so that the same representation also encodes how the scene will change.

2.7 Diffusion Models for Action Generation

Denoising diffusion probabilistic models (DDPMs) learn to generate data by reversing a gradual noising process. The forward process corrupts a clean sample x_0 into x_k according to

$$q(x_k | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_k} x_0, (1 - \bar{\alpha}_k) I), \quad (2.17)$$

where $\bar{\alpha}_k$ follows a fixed noise schedule. A network ϵ_θ is trained to predict the injected noise, $\mathcal{L} = \mathbb{E}_{x_0, \epsilon, k} [\|\epsilon - \epsilon_\theta(x_k, k)\|^2]$, and new samples are generated by iteratively denoising pure Gaussian noise. **Denoising Diffusion Implicit Models** (DDIM) define a non-Markovian sampling process that produces high-quality samples in far fewer steps.

In robotics, **Diffusion Policy** applies this machinery to action sequences: the “data” x_0 is a short sequence of future end-effector poses, and the denoiser is conditioned on the current observation. Diffusion policies naturally represent multi-modal action distributions (for example, going around an obstacle on either side), which is difficult for regression-based policies. GAF uses a diffusion model conditioned on renderings of candidate gripper poses (following Render-and-Diffuse) to refine its initial action estimate,

and at inference time it needs only three DDIM iterations.

2.8 Foundation Models as World Knowledge

Large Language Models (LLMs) are trained on internet-scale text and encode a large amount of commonsense knowledge about the world—for example, that beds are usually found in bedrooms, that bedrooms are often off a hallway, and that refrigerators are in kitchens. **Vision-Language Models** (VLMs) extend this capability to images by aligning visual features with language, either through contrastive embedding models or through generative multimodal models that can answer free-form questions about an image. Recent work has therefore asked whether such models can act as *abstract world models*: rather than simulating physics, they predict the likely outcome of a decision at a semantic level (“if I go through that door, I will probably find a bedroom”).

Foundation models offer three advantages for embodied world modeling: zero-shot generalization to new object categories and environments, no task-specific training data, and the ability to explain their predictions in natural language. They also have well-known weaknesses. They may **hallucinate**—confidently describing objects or relations that are not present—they are relatively slow and expensive to query, and they struggle with precise metric reasoning such as estimating distances in pixels. WMNav’s design choices (quantitative scoring, subtask feedback, and a two-stage action proposer) are directly aimed at these weaknesses.

2.9 Sim-to-Real Transfer

Training in simulation is fast, safe, and cheap, but policies and models trained in simulation must ultimately operate on real hardware. The **sim-to-real gap** arises from unmodeled dynamics (friction, compliance, actuator delays), sensor differences (noise and artifacts in real depth images), and visual appearance. Several techniques are used to bridge it:

- **Domain randomization**: physical parameters (mass, friction, motor strength) and sensor properties are randomized during training so that the real world appears as just another sample from the training distribution.

- **Latency and noise modelling:** sensing and computation delays are simulated explicitly; for example, WMP delivers depth images to the policy with a 100 ms delay in simulation.
- **Privileged learning:** a teacher policy with access to ground-truth simulator information is trained first, and a student policy that sees only onboard sensors is trained to imitate it.
- **Asymmetric actor–critic:** the critic receives privileged information during training, while the actor uses only deployable observations.
- **Real-to-sim-to-real:** a digital twin of the real object or scene is reconstructed, behaviors are planned or learned on the twin, and the result is executed on the real system. This is the strategy of DexSim2Real².

2.10 Evaluation Metrics

Because the reviewed works target different tasks, they are evaluated with different metrics. The most important are summarized below; they are referred to throughout Chapters 3 and 4.

Manipulation error. For goal-conditioned articulated manipulation, the real joint movement $\Delta s_{\text{real}} = s_{\text{real}} - s_{\text{initial}}$ is compared with the target movement $\Delta s_{\text{target}} = s_{\text{target}} - s_{\text{initial}}$, giving an absolute error $\delta = \Delta s_{\text{real}} - \Delta s_{\text{target}}$ (in degrees or centimetres) and a relative error $\delta_r = \delta / \Delta s_{\text{target}} \times 100\%$ [1].

Success rate. The fraction of trials that complete the task, used for RLBench manipulation, real-robot tasks, and legged traversal of obstacles of increasing difficulty.

Reconstruction quality. The Peak Signal-to-Noise Ratio (PSNR), computed as $10 \log_{10}(\text{MAX}^2/\text{MSE})$ decibels, measures photometric fidelity; the Structural Similarity Index (SSIM) measures structural agreement; and the Learned Perceptual Image Patch Similarity (LPIPS) measures perceptual distance using deep features (lower is better).

Locomotion metrics. The average RL return, and the velocity tracking error, defined as the mean squared error between the commanded planar velocity v_{xy}^{cmd} and the achieved velocity v_{xy} [3].

Navigation metrics. Success Rate (SR) and Success weighted by (normalized inverse) Path Length (SPL):

$$\text{SPL} = \frac{1}{N} \sum_{i=1}^N S_i \frac{\ell_i}{\max(p_i, \ell_i)}, \quad (2.18)$$

where $S_i \in \{0, 1\}$ indicates success in episode i , ℓ_i is the shortest-path distance from start to goal, and p_i is the length of the path actually taken. SPL rewards agents that not only succeed but do so efficiently.

2.11 Taxonomy of World Models in Embodied AI

Combining the concepts above, world models used in embodied AI can be organized into four families according to how they represent the world and how they are used for control. Figure 2.11.1 places the four reviewed works in this taxonomy, and Table 2.2 summarizes the defining characteristics of each family.

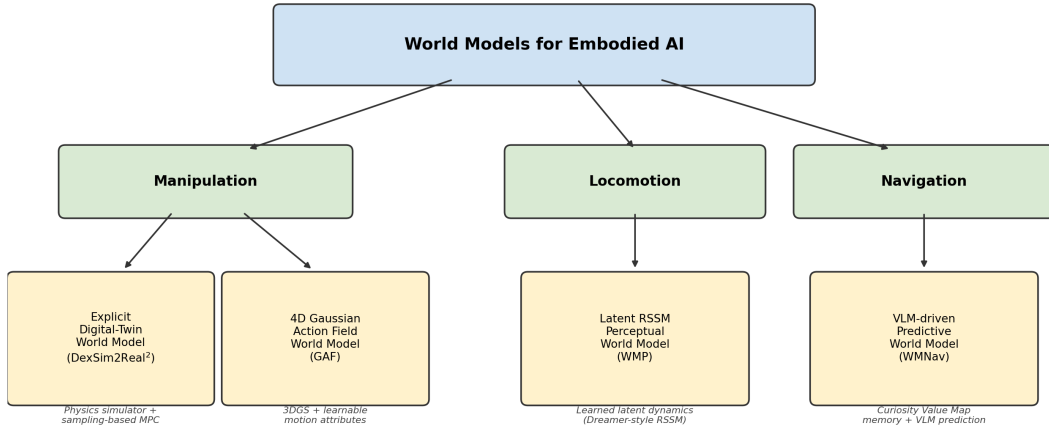


Figure 2.11.1: Taxonomy of world-model approaches in embodied AI

Table 2.2: Families of world models and their characteristics

Family	Representation	Typical use	Reviewed work
Explicit physics-based	Digital twin (meshes, joints, URDF) in a physics engine	Sampling-based MPC, trajectory search, tool use	DexSim2Real ² [1]
Explicit geometric (4D)	3D Gaussians with motion attributes	Future-scene prediction, action extraction and refinement	GAF [2]
Latent (learned)	Recurrent deterministic + stochastic latent state (RSSM)	Perception for a learned policy, imagination-based RL	WMP [3]
Semantic / foundation-model	Language-grounded predictions stored in a value map	Zero-shot high-level planning and exploration	WMNav [4]

2.12 Chapter Summary

This chapter introduced the concepts needed to understand the four reviewed systems. Embodied control was formulated as a POMDP, in which the agent must act on partial observations and therefore benefits from an internal model that summarizes history and predicts the future. World models were defined through their representation, transition, observation, and reward components, and the two main ways of using them—planning at decision time and learning policies from predicted experience—were contrasted. The remaining sections described the specific tools on which each reviewed paper builds:

- **POMDPs, PPO, and the RSSM** provide the formulation, the policy optimizer, and the latent world model of WMP [3].
- **Sampling-based MPC, iCEM, and eigengrasps** provide the planner and the action-space reduction of DexSim2Real² [1].
- **3D Gaussian Splatting and diffusion models** provide the scene representation and the action refinement of GAF [2].
- **Foundation models** provide the commonsense predictor at the heart of WMNav [4].
- **Sim-to-real techniques and metrics** explain how the systems are validated.

Chapter 3

Literature Survey

This chapter reviews four selected research papers under the theme of **World Models for Embodied AI**. Each work instantiates the world-model paradigm in a different embodied domain and with a different representation: an explicit digital-twin world model for precise articulated object manipulation, a 4D Gaussian Action Field for dynamic manipulation scenes, a latent recurrent world model for visual legged locomotion, and a VLM-driven predictive world model for zero-shot object goal navigation. Together, these studies illustrate the evolution of embodied agents from reactive perception–action mappings toward predictive, simulation-capable intelligence. For each paper, the problem setting, architecture, key equations, experimental protocol, and principal results are presented, followed by a short critical discussion.

3.1 DexSim2Real²: Building Explicit World Model for Precise Articulated Object Dexterous Manipulation

This paper, the base paper of the seminar, proposes a robot learning framework for precise, goal-conditioned manipulation of unseen articulated objects using suction grippers, two-finger grippers, and multi-finger dexterous hands [1]. Its core idea is to use a physics simulator as the robot’s “mental model”: the robot first interacts with the object to reveal its structure, then constructs an explicit digital twin of the object in simulation, and finally plans long-horizon manipulation trajectories inside this world model using sampling-based Model Predictive Control (MPC), without demonstrations or reinforcement learning. The work extends the authors’ earlier ICRA paper Sim2Real².

3.1.1 Problem Statement and Motivation

Articulated objects—drawers, cabinets, laptops, lamps, refrigerators—are ubiquitous in daily life, and manipulating them is considerably harder than pick-and-place. In pick-and-place only the start and end poses of the end effector are constrained, whereas articulated object manipulation requires the end effector to follow a trajectory dictated by the object’s kinematic constraint. Most existing approaches train a neural network with RL or IL to map object states to actions, but the state distribution of articulated objects is high-dimensional and complex, so such networks struggle to learn the correlation even with hundreds of demonstrations and millions of interactions. Methods such as FlowBot and UMPNet predict one-step actions from a single frame and are tied to suction grippers.

DexSim2Real² addresses the *goal-conditioned* version of the problem: given an unseen articulated object and a target joint state s_{target} , drive the object precisely to that state with any of several end effectors. The key observation is that a single view of an object cannot reveal its full structure—it is hard to tell from one look whether a kitchen door has a rotating or a sliding hinge—but after the door is moved slightly, the two observations together make the structure obvious. The framework therefore separates the problem into a *generalizable* part (predicting a simple interaction and building a model) and an *instance-specific* part (planning with the model of the one object in front of the robot).

3.1.2 Framework Overview

As shown in Figure 3.1.1, the framework consists of three phases. (1) In **Interactive Perception**, an affordance prediction module proposes a one-step interaction from a partial point cloud of the object; the robot executes it to change the object’s joint state and captures new observations, repeating this K times for an object with K movable parts. (2) In **Explicit World Model Construction**, the $K + 1$ frames of observations are used to build a digital twin—part meshes plus kinematic structure—that can be loaded into a physics simulator. (3) In **Sampling-based Model Predictive Control**, a long-horizon trajectory is searched in simulation and then executed on the real robot; for dexterous hands, an eigengrasp module first reduces the action dimension.

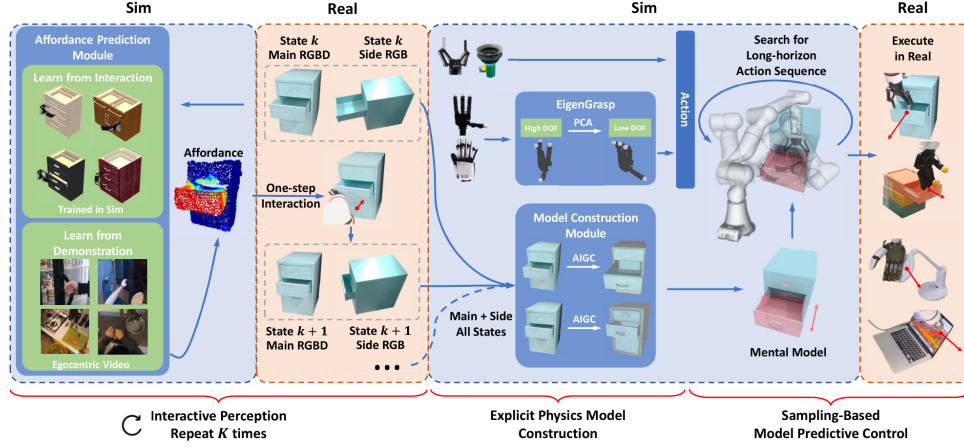


Figure 3.1.1: Overview of the DexSim2Real² framework [1]

3.1.3 Interactive Perception

The purpose of interactive perception is only to *change* the state of each movable part; the interaction does not need to be precise. Affordance estimation depends only on the object and therefore generalizes better to novel objects than a full manipulation policy. Two sources of affordance knowledge are studied.

Learning from self-supervised interaction in simulation. The framework uses Where2Act, which contains an Actionability Scoring Module that predicts a per-point score a_p (the likelihood that acting at that point moves a part), an Action Proposal Module that proposes gripper poses for a point, and an Action Scoring Module that predicts the success likelihood of each proposal. Because Where2Act assumes a free-flying gripper and ignores the robot’s kinematics, many top-scoring proposals are unreachable. DexSim2Real² therefore selects the n_p points with the highest actionability, keeps the n_a best actions for each, and attempts motion planning sequentially until a feasible action is found ($n_p = n_a = 10$ in the experiments).

Learning from real-world egocentric human videos. Simulation-based affordance learning requires interactable 3D assets, which are scarce. The authors therefore also use the Vision-Robotics Bridge (VRB), trained on videos of humans manipulating objects, which predicts a 2D contact heatmap and a 2D post-contact trajectory vector in the image. To lift these 2D predictions into a 3D robot motion, a novel **pixel projection**

method is proposed (Figure 3.1.2). Given the RGB image I_0 , depth map D_0 , camera intrinsics K and extrinsics H_0 , a virtual camera with pose H is placed and a virtual image $\hat{I}_0$ is synthesized by image warping. VRB is run on both images, producing contact points $c_0 = (u_0, v_0)$, $\hat{c}_0$ and post-contact vectors $\tau_0 = (u'_0 - u_0, v'_0 - v_0)$, $\hat{\tau}_0$. The 3D contact point in the robot base frame is

$$p_c = H_0^{-1} K^{-1} z_c [u_0, v_0, 1]^\top, \quad (3.1)$$

where z_c is the depth of the contact point. Each 2D vector is the projection of infinitely many 3D vectors that lie in a common *projection plane*. For the real camera this plane is spanned by

$$\varphi_0 = p'_c - p_c, \quad \varphi'_0 = p_c - o_{c_0}, \quad p'_c = H_0^{-1} K^{-1} z_c [u'_0, v'_0, 1]^\top, \quad (3.2)$$

where o_{c_0} is the camera origin in the robot frame. The plane normal is $n_0 = \varphi_0 \times \varphi'_0$, the normal $\hat{n}_0$ of the virtual camera's plane is computed in the same way, and the 3D post-contact direction is the intersection of the two planes:

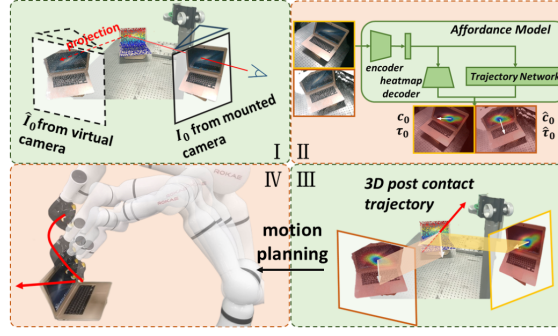
$$\tau_c = n_0 \times \hat{n}_0. \quad (3.3)$$

The robot then moves its end effector to p_c and pushes a fixed distance along τ_c . For fully closed cabinets and drawers where pushing is infeasible, VRB is used to detect the handle, the gripper is placed behind it, closed, and pulled along τ_c . For objects with $K > 1$ movable parts, affordance points are detected iteratively by masking the neighbourhood of each found point.

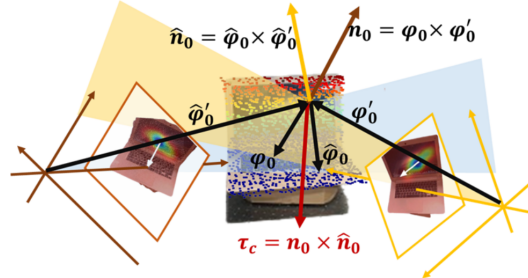
3.1.4 Explicit World Model Construction

The second module builds the digital twin from the $K + 1$ observation frames

$$\{I_0^{(k)}, D_0^{(k)}, M_0^{(k)}, I_1^{(k)}, M_1^{(k)}, \dots, I_C^{(k)}, M_C^{(k)}\}_{k=0}^K, \quad (3.4)$$



(a)



(b)

Figure 3.1.2: (a) Generating a real robot manipulation trajectory from 2D affordances; (b) computing the 3D post-contact vector as the intersection of two projection planes [1]

where I_0, D_0 come from the main RGB-D sensor, I_c from C additional uncalibrated RGB cameras ($C = 1$ in the experiments), and M are object masks. The pipeline, shown in Figure 3.1.3, has four stages.

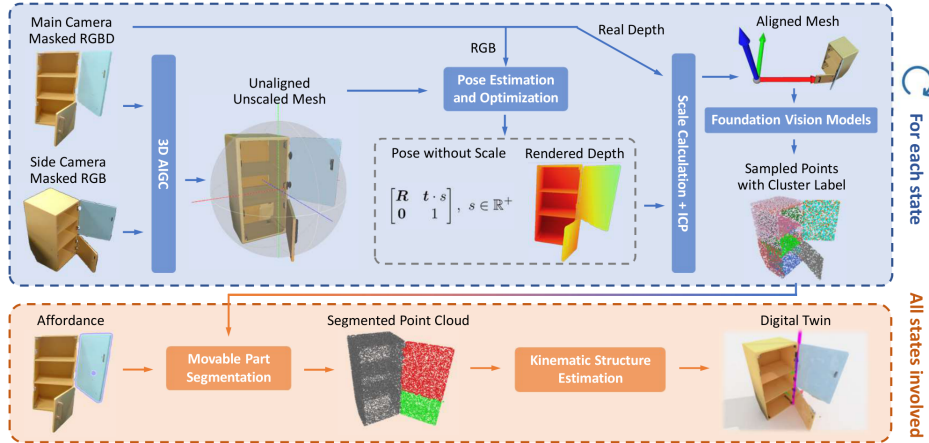


Figure 3.1.3: Explicit world model construction pipeline: 3D AIGC mesh generation, pose and scale estimation, movable part segmentation, and kinematic structure estimation [1]

1) Canonical mesh generation. For each state k , a controllable large-scale 3D generative model, Rodin, generates a textured canonical mesh $\tilde{A}^{(k)}$ conditioned on the masked multi-view images. Using 3D AIGC rather than direct depth fusion yields complete, high-quality geometry even for regions that are not visible from the main camera.

2) Mesh positioning and scaling. The generated mesh is centred at the origin and normalized to unit size, so standard CAD-based pose estimators cannot be applied directly. The authors modify FoundationPose: images of $\tilde{A}^{(k)}$ are rendered under many pose hypotheses and ranked by similarity to $I_0^{(k)}$, and the best hypothesis is refined by aligning rendered edges to the mask $M_0^{(k)}$ using differentiable rendering, with the orientation constrained to rotate about the Z-axis for stability. Given the orientation and unscaled translation, an unscaled depth map $\tilde{D}_0^{(k)}$ is rendered and the metric scale is recovered as the ratio of real to rendered depth over the object mask:

$$s^{(k)} = \frac{\sum_{M_0^{(k)}} D_0^{(k)}}{\sum_{M_0^{(k)}} \tilde{D}_0^{(k)}}. \quad (3.5)$$

3) Movable part segmentation. Distinguishing parts from a single static mesh is difficult, so all $K + 1$ meshes are used jointly. Each mesh is first split into small sub-parts using the foundation 3D segmentation model SAMPart3D. For each sub-part, bidirectional nearest-neighbour (chamfer-style) distances to the previous frame’s point cloud are computed; a sub-part is considered moved if these distances are large or their distributions differ. To make this robust to inconsistencies of 3D AIGC and pose errors, two further cues are used: *mesh connectivity*, encoded as a graph $G^{(k)} = \langle Q^{(k)}, E^{(k)} \rangle$ over sub-parts, and *robot proprioception*—the sub-part closest to the end effector during interaction must have moved. The graph is traversed breadth-first from this sub-part, and a sub-part is labelled as moved only if at least one predecessor is moved. Algorithm 2 summarizes the procedure.

4) Kinematic structure construction. Finally, the part tree hierarchy, joint categories (prismatic or revolute), and joint configurations (axis direction and origin) are estimated following Liu et al.: per-part rigid transformations between frames are estimated

Algorithm 2 Movable part segmentation in DexSim2Real² (adapted from [1])

Require: meshes $\{A^{(k)}\}_{k=0}^K$ **Ensure:** movable part segmentation $S^{(K)}$

```

1: for  $k = 0, \dots, K$  do
2:   Segment  $A^{(k)}$  into sub-parts  $Q^{(k)}$  with SAMPart3D
3:   Build graph  $G^{(k)} = \langle Q^{(k)}, E^{(k)} \rangle$ ; find sub-part  $Q_{\text{closest}}$  nearest the end effector
4:   Sample point cloud  $P^{(k)}$  with sub-part indices from  $A^{(k)}$ 
5: end for
6:  $S^{(0)} \leftarrow 0$ 
7: for  $k = 1, \dots, K$  do
8:   for each sub-part  $Q$  in breadth-first order of  $G^{(k)}$  from  $Q_{\text{closest}}$  do
9:      $P \leftarrow P^{(k)}[x]$  for the point indices  $x$  of  $Q$ 
10:    Compute distances  $\{d_i^{(k) \rightarrow (k-1)}\}$  from  $P$  to its closest points  $P^B$  in  $P^{(k-1)}$ 
11:    Compute distances  $\{d_i^{(k-1) \rightarrow (k)}\}$  from  $P^B$  back to  $P^{(k)}$ 
12:    if distances indicate motion and a predecessor of  $Q$  is moved then
13:       $S^{(k)}[x] \leftarrow k$ 
14:    else
15:       $S^{(k)}[x] \leftarrow S^{(k-1)}[x_{\text{closest}}]$ 
16:    end if
17:  end for
18: end for
19: return  $S^{(K)}$ 

```

and then projected onto a valid kinematic model. The result is exported in the Unified Robot Description Format (URDF), the meshes are convex-decomposed with VHACD for stable contact simulation, and the twin is loaded into the SAPIEN simulator. Compared with Ditto, this pipeline handles objects with more than one movable part and produces significantly higher-quality geometry.

3.1.5 Sampling-Based MPC with Eigengrasp

With the digital twin in hand, the agent must find a trajectory that changes the joint state from s_{initial} to s_{target} , i.e. achieve $\Delta s_{\text{target}} = s_{\text{target}} - s_{\text{initial}}$. Because contacts between the end effector and the object make the objective non-smooth, the zeroth-order iCEM planner (Section 2.5) is used. At each time step $t < T$ the action $a_t \in \mathbb{R}^d$ is an increment of the robot's joint positions, where $d = 7$ for the suction gripper (7-DOF arm), $d = 8$ for the two-finger gripper (arm plus gripper), and $d = 7 + d_{\text{hand}}$ for a dexterous hand. The population, planning horizon h , and elite set size E control the search.

For dexterous hands, searching directly in the $7 + 16 = 23$ -dimensional space of the Allegro Hand is computationally prohibitive and produces unnatural postures. The authors therefore generate a dataset of grasp postures with DexGraspNet—60,800 grasps across 474 objects for the Allegro Hand and the XHand—perform PCA, and keep the first m eigenvectors $e_1, \dots, e_m$. iCEM then searches over $a_t \in \mathbb{R}^{7+m}$ and the hand joint angles are recovered as

$$q_h = \sum_{i=1}^m a_i e_i. \quad (3.6)$$

Figure 3.1.4 shows that the accumulated variance ratio of the two hands follows nearly the same curve despite their different numbers of joints, suggesting that eigengrasps capture fundamental features of hand posture; $m = 2$ is used unless stated otherwise.

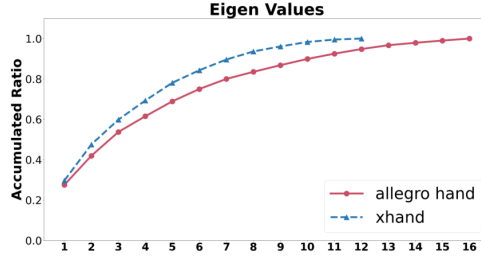


Figure 3.1.4: Accumulated variance ratio versus eigengrasp dimension for the Allegro Hand and the XHand [1]

Dense reward design. To guide the search efficiently, dense rewards are defined for each end effector. Common to all are a success reward and an approaching reward:

$$r_{\text{success}} = \begin{cases} \omega_s, & |s_{\text{target}} - s_t| < \epsilon \\ 0, & \text{otherwise} \end{cases}, \quad r_{\text{target}} = -\omega_t \frac{|s_{\text{target}} - s_t|}{|s_{\text{target}} - s_{\text{initial}}|}. \quad (3.7)$$

A contact reward encourages correct, sustained contact and penalizes collisions:

$$r_{\text{contact}} = \begin{cases} \omega_{\text{contact}}, & \frac{|s_t - s_{\text{target}}|}{|s_{\text{target}} - s_{\text{initial}}|} < 1 \\ -\omega_{\text{collision}}, & \text{unexpected collision} \\ 0, & \text{otherwise.} \end{cases} \quad (3.8)$$

A distance term r_{dist} , weighted by ω_d and computed from $\|p_{\text{part}} - p_{\text{grasp}}\|_2^2$, draws the end effector (or, for suction, a virtual link at the suction tip) towards the geometric centre of the target part. The suction gripper additionally receives an access-direction reward $r_{\text{dir}} = \omega_{\text{dir}}$ when the angle between the suction axis and the target surface normal is within a threshold ψ . The two-finger gripper uses a regularization reward $r_{\text{reg}} = -\sum_{i=1}^d (\omega_a a_i + \omega_v v_i)$ that penalizes joint accelerations and velocities. For the dexterous hand, the contact reward is granted only when the palm *and* at least two fingers touch the object, encouraging the hand to cage the part, and the regularization $r_{\text{reg}} = -\omega_p e_p - \sum_i \omega_v v_i$ also penalizes the Cartesian error e_p of the end link.

3.1.6 Experimental Setup

Real-world experiments use a 7-DOF ROKAE xMate3Pro arm fixed to a table, an Intel RealSense D415 RGB-D camera (the only sensor that requires calibration), and an additional side RGB camera. Four end effectors are evaluated: a suction gripper, a Robotiq 2F-140 two-finger gripper, a 16-DOF four-finger Allegro Hand, and a 12-DOF five-finger XHand. Test objects come from four categories—drawers, cabinets, lamps, and laptops—placed at random positions with random initial and goal joint states covering both directions of motion. Joint states after manipulation are measured with a double-arm protractor. SAPIEN is used both to train the affordance network and as the MPC simulator. The planner runs 30 sampling processes on a workstation with an AMD Ryzen 9 9950X CPU and an NVIDIA RTX 3080Ti GPU. Table 3.1 lists the planning parameters.

Table 3.1: Sampling-based MPC settings used in DexSim2Real² [1]

End effector	Action dim. d	T	E	h	Search time
Suction gripper	7	50	20	10	≈ 1.5 min
Two-finger gripper (Robotiq 2F-140)	8	50	300	10	≈ 2.5 min
Dexterous hand (Allegro, $m = 2$)	$7 + m = 9$	50	100	10	≈ 3 min

Joint increments are limited to $[-0.05, 0.05]$ per step. For the suction gripper the reward weights are $\omega_s = 20$, $\omega_t = 50$, $\omega_{\text{contact}} = 10$, $\omega_{\text{collision}} = 60$, $\omega_d = 10$, $\omega_a = 0.01$, $\omega_v = 0.03$, $\psi = 15^\circ$, with success thresholds $\epsilon = 0.005$ for prismatic and 0.02 for revolute joints. For VRB-based affordance, the virtual camera is placed at $(0.2, 0, 0.7)$ in the robot base frame

with a pitch of 52° , balancing visual similarity against the baseline required for accurate plane intersection.

3.1.7 Results and Analysis

Suction gripper versus UMPNet. UMPNet, which also performs goal-conditioned articulated manipulation, is used as the baseline on laptops and cabinets with ten trials each. Table 3.2 and Figure 3.1.5(a) show that DexSim2Real² roughly halves the error: 7.2° versus 16.0° on laptops and 6.8° versus 15.3° on cabinets. The authors attribute UMPNet’s larger error to its fixed step size and to the limited information in a pair of current/goal images, whereas MPC over an explicit model uses continuous actions.

Table 3.2: Accuracy of real articulated object manipulation with the suction gripper [1]

Metric	Laptop		Cabinet	
	UMPNet	Ours	UMPNet	Ours
Trials with $ \delta_r < 10\%$ ($\uparrow$)	0/10	3/10	0/10	4/10
Trials with $ \delta_r < 30\%$ ($\uparrow$)	3/10	9/10	5/10	7/10
Trials with $ \delta_r < 50\%$ ($\uparrow$)	7/10	9/10	6/10	8/10
Average $ \delta $ ($\downarrow$)	16.0°	7.2°	15.3°	6.8°
Average $ \delta_r $ ($\downarrow$)	42.9%	23.3%	41.5%	21.7%

Two-finger gripper. Twenty trials each on laptops and cabinets (Table 3.3) yield average errors of 7.2° and 7.4° , with about 75% of manipulations achieving $|\delta_r| < 30\%$. A reward ablation on four tasks (opening/closing a drawer and a laptop) shows that the full reward achieves the best success rate and fewest steps, except when r_{reg} is removed; however, trajectories found without r_{reg} drive the robot into unusual and potentially dangerous configurations. Without r_{dist} the task cannot be completed at all, because the short planning horizon never reaches positive reward, and removing r_{target} , r_{success} , or r_{contact} lowers success and lengthens trajectories.

Dexterous hand. Using the Allegro Hand with $m = 2$, twenty trials each on drawers and cabinets give an average error of 1.64 cm for prismatic drawers and 6.87° for revolute cabinets (Table 3.3). All forty manipulations achieve $|\delta_r| < 50\%$.

Table 3.3: Accuracy of real manipulation with the two-finger gripper and the dexterous hand [1]

Metric	Two-finger gripper		Dexterous hand	
	Laptop	Cabinet	Drawer	Cabinet
Number of manipulations	20	20	20	20
$ \delta_r < 10\%$	6	4	3	3
$ \delta_r < 30\%$	16	15	11	18
$ \delta_r < 50\%$	20	18	20	20
Average $ \delta $	7.2°	7.4°	1.64 cm	6.87°
Average $ \delta_r $	21.5%	22.5%	26.4%	20.4%

Effect of eigengrasp dimension. Simulation experiments with $m \in \{1, 2, 7, 16\}$ on four tasks (ten randomized repetitions each) evaluate success rate, joint jerk, and computation time. Opening a laptop and closing a drawer succeed 100% of the time for all m , but closing a laptop and opening a drawer fail more often with $m = 1$ because the hand loses dexterity. Crucially, $m = 2$ performs comparably to $m = 7$ and $m = 16$, while finger jerk grows with m and $m = 2$ saves roughly one second per planning step compared with $m = 16$ —nearly one minute per trajectory. A dexterous-hand reward ablation shows the same pattern as for the gripper, with the interesting exception that removing r_{contact} sometimes shortens trajectories, because the planner is then free to explore unconventional, non-human-like hand postures.

Dexterous hand versus two-finger gripper. On five simulated tasks with ten randomized initial conditions each, the Allegro Hand requires fewer steps than the Robotiq gripper on every task except laptop opening, which does not require precise contact. For example, the dexterous hand opens a drawer in 17 steps versus 22 for the gripper by exploiting its additional degrees of freedom.

World model construction accuracy. The digital twins are evaluated with the Relative Depth Error (RDE) between rendered and real depth, the IoU between rendered and ground-truth masks, and the joint axis position and angle errors. Table 3.4 and Figure 3.1.5(b) compare the axis angle error with ScrewNet. The average angle error is below 9° for all four categories, far lower than ScrewNet on laptops (1.34° vs. 23.15°) and drawers (8.58° vs. 23.21°). The lower IoU for lamps is attributed to their small image

footprint. Deliberately corrupting the model shows that manipulation error correlates positively with modelling error, with joint *position* errors having a stronger effect than orientation errors.

Table 3.4: Explicit world model construction accuracy [1]

Category	RDE (%)	IoU	Pos. err. (m)	Angle err. (°)	ScrewNet angle err. (°)
Laptop	1.47	0.92	0.022	1.34	23.15
Cabinet	3.77	0.94	0.029	4.84	11.67
Drawer	2.23	0.94	–	8.58	23.21
Lamp	2.12	0.80	0.009	7.95	9.38

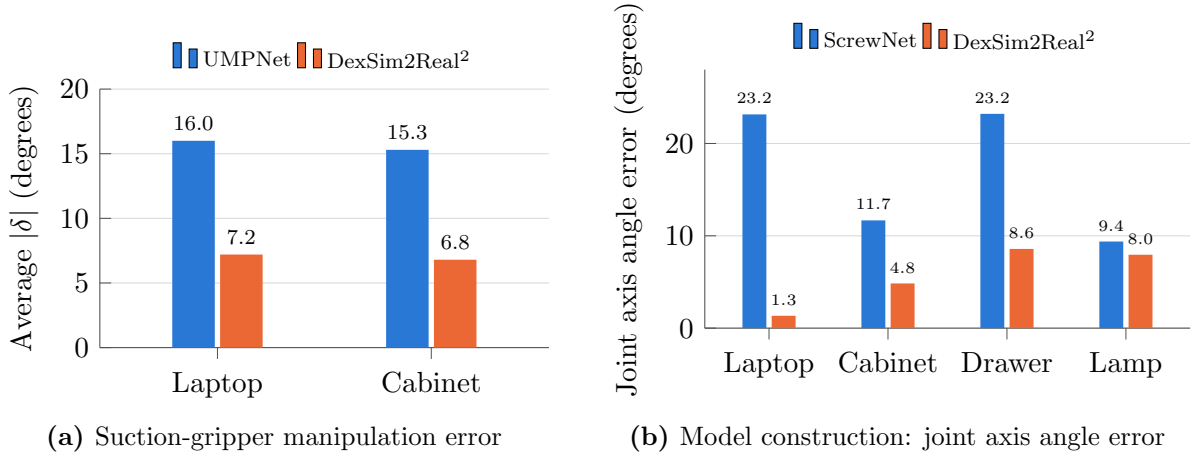


Figure 3.1.5: Quantitative results of DexSim2Real² plotted from the values reported in [1] (lower is better)

Ablation of movable part segmentation and affordance. Without the proposed segmentation module (i.e. jointly optimizing segmentation and motion as in the method of Liu et al.), model construction succeeds on only 5/10 drawers instead of 10/10 and the drawer axis error doubles from 8.58° to 16.04°; position errors also increase for laptops (0.022 to 0.048 m), cabinets, and lamps, and the baseline sometimes produces the wrong number of parts or wrong joint types. For affordance detection on closed objects, VRB finds handles on 17/20 cabinets versus 7/20 for Where2Act, and on 3/20 versus 1/20 small drawer handles; in over 90% of real-world cases VRB proposes the better action, which the authors attribute to its training on real human videos and to the larger scale of video

data compared with interactable 3D assets. The pixel projection method also produces better interaction directions than random 3D vectors consistent with the 2D affordance.

Multi-part objects and tool use. Figure 3.1.6(a) shows reconstructed twins of a cabinet with two revolute doors and a refrigerator with two revolute joints and one prismatic joint, demonstrating that the pipeline handles varied kinematic structures. Because the world model is a physics simulator, it generalizes to actions never observed during interaction: when a drawer is out of reach or its gap is too small for the gripper, the robot can use a T-shaped tool or a semi-ring (Figure 3.1.6(b)). The only change needed is to replace the gripper tips with the tool in r_{dist} ; most other parameters are unchanged.

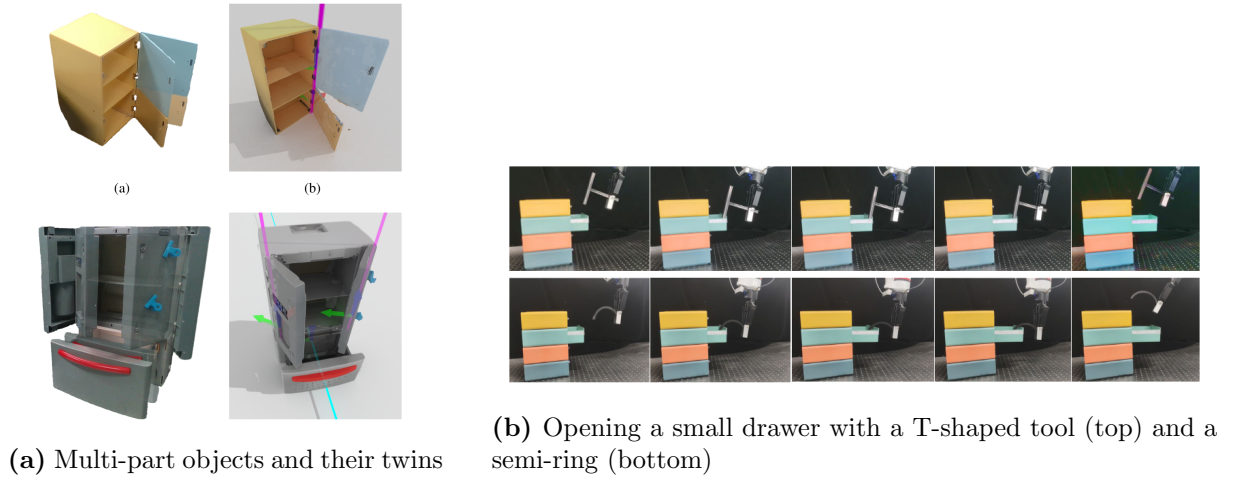


Figure 3.1.6: Generalization of the explicit world model: (a) reconstructed twins of a cabinet with two revolute joints and a refrigerator with two revolute joints (pink) and one prismatic joint (blue); (b) tool use planned entirely in the world model [1]

3.1.8 Discussion

By first building the world model of a single object instance and then planning within it, the framework avoids the data-hungry generalization burden of policy networks. The explicit physics prior guarantees that a model constructed from simple one-step interactions supports long-horizon, multi-step, and even unseen tool-using trajectories, marking a distinctive advantage over articulation-flow and policy-based baselines. The authors identify two sources of residual error beyond modelling inaccuracy: real objects have complex dynamics (a laptop lid deforms elastically and partially springs back when released), and real joints are not ideal (rail gaps turn a prismatic joint into a joint with several degrees

of freedom). The method also requires every movable part to change observably during interactive perception, and 3D AIGC may produce misaligned meshes for novel or uncommon objects. The authors propose extending the framework to scene-level digital twins for long-horizon mobile manipulation and generating rewards from videos.

3.2 GAF: Gaussian Action Field as a 4D Representation for Dynamic World Modeling in Robotic Manipulation

This research introduces a Vision-to-4D-to-Action (V-4D-A) paradigm in which manipulation actions are reasoned directly from a motion-aware 4D scene representation [2]. The Gaussian Action Field (GAF) extends 3D Gaussian Splatting (3DGS) with a learnable motion attribute per Gaussian, so that a single feed-forward network simultaneously reconstructs the current scene, predicts the future scene, and yields an initial estimate of the robot action from the predicted Gaussian motion.

3.2.1 Problem Statement and Motivation

Vision-based manipulation methods fall into two paradigms (Figure 3.2.1). **V-A** (vision-to-action) methods such as Diffusion Policy map images directly to actions; they benefit from end-to-end learning and large image datasets but understand 3D structure only implicitly and struggle in high-precision tasks. **V-3D-A** (vision-to-3D-to-action) methods such as Act3D lift observations to point clouds or voxels, capturing spatial relationships more accurately, but require 3D data that is scarcer than images. Both share a deeper limitation: they model a *static* snapshot of the scene, even though manipulation is inherently about how the scene—including the robot itself—will change. This creates a mismatch between static scene understanding and dynamic action generation. Prior Gaussian world models such as ManiGaussian and GWM deform Gaussians to predict the future, but use the prediction only as auxiliary supervision for policy training, and their low-fidelity reconstructions limit effectiveness.

3.2.2 The Gaussian Action Field Representation

GAF is defined as a unified spatiotemporal representation that associates each Gaussian primitive $g(x)$ at time t with both geometric attributes and motion dynamics, parame-

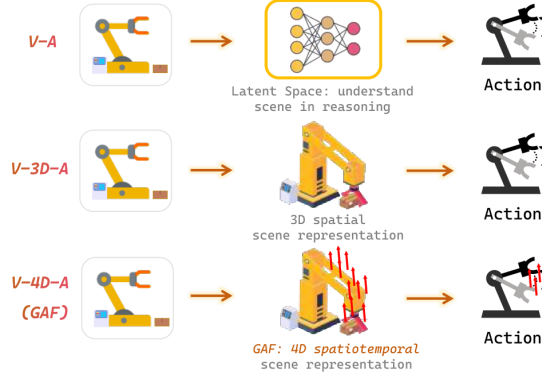


Figure 3.2.1: Comparison of the V-A, V-3D-A, and the proposed V-4D-A (GAF) paradigms [2]

terized by a continuous function

$$\mathcal{F}_{\Theta} : \{g(x), t\} \mapsto \{\mu, \Delta\mu, f\}, \quad (3.9)$$

where $\mu \in \mathbb{R}^3$ is the Gaussian’s position, $\Delta\mu \in \mathbb{R}^3$ its displacement over a future interval, and $f = \{c, \sigma, r, s\}$ its colour, opacity, rotation, and scale. Three outputs are derived: the **current Gaussians** $\{\mu, f\}$ representing the present scene; the **future Gaussians** $\{\mu + \Delta\mu, f\}$ representing how the scene will evolve; and the **init action**, obtained by registering the current and future Gaussians that belong to the manipulator.

3.2.3 Architecture: Dynamic Gaussian Reconstruction

GAF adopts a geometry-agnostic, *pose-free* reconstruction approach. Unlike NeRF or classical 3DGS, which require dense calibrated views or geometric priors such as cost volumes, GAF reconstructs motion-augmented Gaussians from only two unposed RGB images and their intrinsics $\{I_v^t, k_v^t\}_{v=1}^V$, in a canonical frame aligned with the first view (Figure 3.2.2). The images are split into patches, concatenated, and processed by a shared-weight Vision Transformer with cross-view attention, initialized from MAST3R. Three DPT-style heads decode the features: a Gaussian Center Head h_{center} predicts positions, a Gaussian Params Head h_{feature} predicts appearance, and a Motion Prediction

Head h_{motion} predicts per-point displacements:

$$\mathcal{H}_{\text{Gauss}}(\text{ViT}(\{I_v^t, k_v^t\}_{v=1}^V)) = \{\mu_j^t, c_j^t, \sigma_j^t, r_j^t, s_j^t\}_{j=1}^{V \times H \times W}, \quad (3.10)$$

$$h_{\text{motion}}(\text{ViT}(\{I_v^t, k_v^t\}_{v=1}^V)) = \{\Delta\mu_j^{t \rightarrow t+\Delta t}\}. \quad (3.11)$$

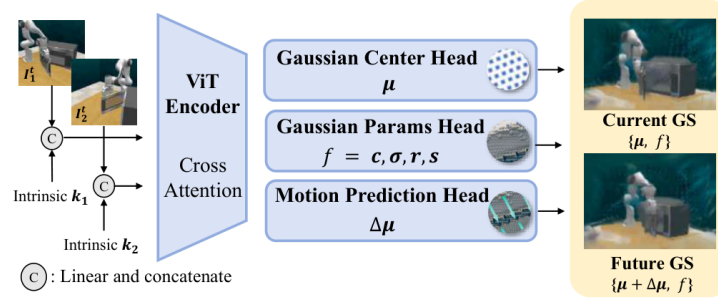


Figure 3.2.2: GAF reconstruction: a ViT with cross-view attention and three heads predicts Gaussian centres, appearance parameters, and motion [2]

Adding the displacements to the centres gives the future Gaussians. Both current and future Gaussians are rendered to M novel views using the alpha-blending rule of Equation (2.16), which allows the whole network to be trained purely from RGB video frames, without depth, camera poses, 3D ground truth, or action labels. The training objective combines perceptual and pixel losses on both time steps:

$$\mathcal{L}_{\text{GAF}} = \mathcal{L}_{\text{LPIPS}}^t + \mathcal{L}_{\text{MSE}}^t + \mathcal{L}_{\text{LPIPS}}^{t+\Delta t} + \mathcal{L}_{\text{MSE}}^{t+\Delta t}, \quad (3.12)$$

where the t terms enforce fidelity to the current observation and the $t + \Delta t$ terms regularize future prediction.

3.2.4 Init Action Computation

Because the gripper is rigid, its motion between the current and predicted future scenes is a rigid transformation. GAF extracts the gripper Gaussians μ_{gripper} and their future counterparts $(\mu + \Delta\mu)_{\text{gripper}}$ and estimates the end-effector transformation $T^{t \rightarrow t+\Delta t} \in$

$SE(3)$ with Iterative Closest Point (ICP):

$$T^{t \rightarrow t+\Delta t} = \arg \min_T \sum_{k \in \text{gripper}} \|T(\mu_k) - (\mu + \Delta\mu)_k\|^2. \quad (3.13)$$

This transformation is interpolated into a sequence over discrete time steps, forming the init action a_{init} . In this way, the action is not predicted by a separate policy head but *read out* of the predicted future geometry.

3.2.5 Manipulation with GAF: Action-Vision-Aligned Denoising

Since GAF is trained only on visual data, its init actions are visually plausible but not necessarily physically feasible. A refinement stage (Figure 3.2.3) therefore uses a diffusion model and a small amount of real action data, following Render-and-Diffuse. For a candidate relative action a , the gripper’s new world pose is $T_{g_{\text{new}}}^w = T_g^w \times a$, and its CAD model is rendered into each calibrated camera c :

$$R_c = \text{Render}(T_{w2c} \times T_g^w \times a, K_c). \quad (3.14)$$

Overlaying these renderings on the current multi-view images produces the **Actionable Multiview RGB Guidance**, which places the visual observation and the candidate action in a single image space. The diffusion model is trained with

$$\mathcal{L}_{\text{refine}} = L_1(D, D_{gt}) + L_1(\epsilon, \epsilon_{gt}) + \text{BCE}(g, g_{gt}), \quad (3.15)$$

where D is the denoising direction of the gripper, ϵ the noise added to the end-effector action, and g the binary gripper open/close state. The denoised poses are executed with inverse kinematics, new observations are collected, and the loop repeats until the task is complete, providing closed-loop robustness to occlusion and interaction uncertainty.

3.2.6 Experimental Setup and Implementation

Simulation experiments use nine RLBench tasks covering fine-grained placement, occlusion-rich interactions, and articulated objects, with 20 demonstrations per task and evaluation on 100 unseen object poses. Baselines are Diffusion Policy (V-A), Act3D

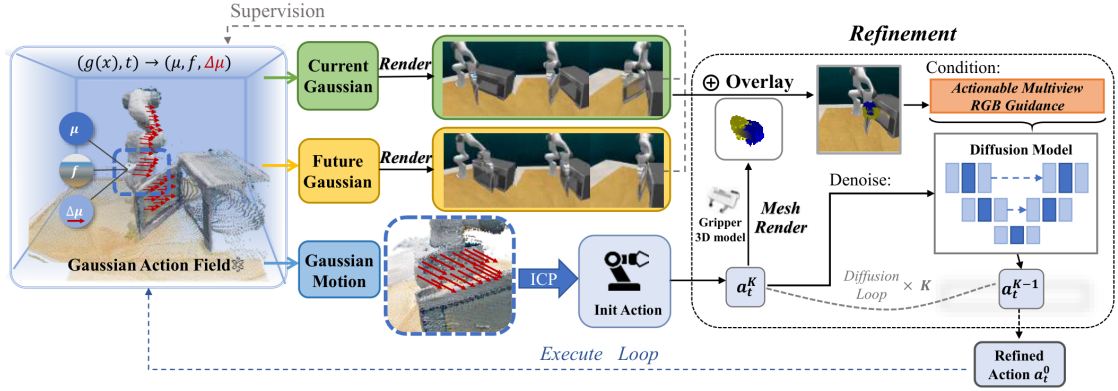


Figure 3.2.3: GAF manipulation pipeline with action-vision-aligned denoising [2]

(V-3D-A), and ManiGaussian (Gaussian world model), with hyper-parameters matched for fairness. All methods use two external cameras and a wrist camera (128×128 in simulation, 1280×720 in the real world). Table 3.5 summarizes implementation details.

Table 3.5: Implementation details of GAF [2]

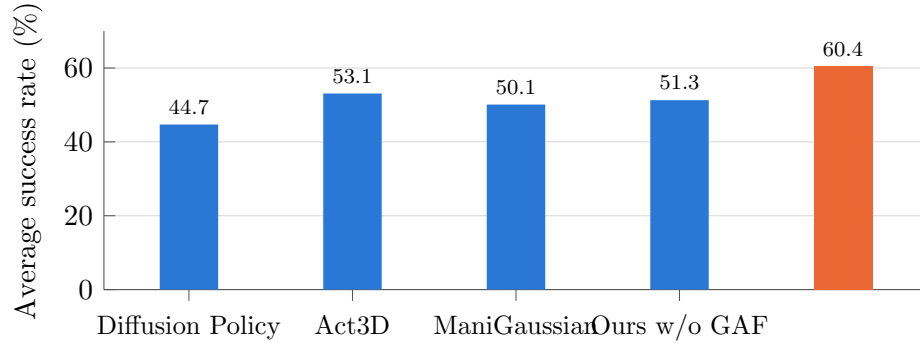
Item	Setting
Reconstruction backbone	ViT with cross-view attention, initialized from MAST3R; DPT heads
GAF training	80k iterations, batch size 16, single NVIDIA A800 GPU, about 24 hours
Diffusion training	50 DDIM iterations; 1 past observation in, 8 future actions out; 50k iterations in about 1.5 days on a single GPU
Inference	3 diffusion iterations; each 8-step action prediction takes less than 0.3 s
Real-world deployment	Prediction and execution run in parallel threads to hide latency

3.2.7 Results and Analysis

Manipulation success on RLBench. Table 3.6 and Figure 3.2.4 show that GAF achieves the highest average success rate of 60.4%, improving on Diffusion Policy by 15.7 points, on Act3D by 7.3 points, and on ManiGaussian by 10.3 points. Gains are largest on occlusion-heavy tasks such as *Toilet Seat Down* (71% versus 39% for DP), highlighting the value of 3D and 4D perception. Removing GAF (predicting actions from images with a diffusion model starting from random noise, without multi-view rendering or init-action priors) reduces the average by 9.1 points to 51.3%.

Table 3.6: Success rates (%) on nine RLBench tasks [2]

Method	Toilet seat down	Open grill	Close grill	Close fridge	Phone on base	Lift lid up	Close microwave	Push button	Close laptop	Average
Diffusion Policy	39	20	36	16	23	81	62	61	64	44.7
Act3D	60	22	41	47	26	91	73	60	58	53.1
ManiGaussian	34	24	38	41	30	97	67	63	57	50.1
Ours w/o GAF	57	16	49	27	31	94	79	58	51	51.3
GAF (ours)	71	26	55	42	35	100	85	61	69	60.4

**Figure 3.2.4:** Average RLBench success rate of GAF (orange) compared with baselines and its own ablation (blue), plotted from [2]

Scene reconstruction and prediction. Compared with ManiGaussian on three tasks, GAF improves current-scene reconstruction by +11.54 dB PSNR, +0.386 SSIM, and -0.557 LPIPS on average, and future-scene prediction by +10.53 dB PSNR, +0.386 SSIM, and -0.576 LPIPS (Table 3.7). Qualitatively, GAF preserves fine structures such as the gripper’s articulated joints even under partial observation, which is exactly what makes ICP-based action extraction possible. Dynamic reconstruction methods such as 4DGS are not compared because they interpolate between observed frames rather than predicting future states.

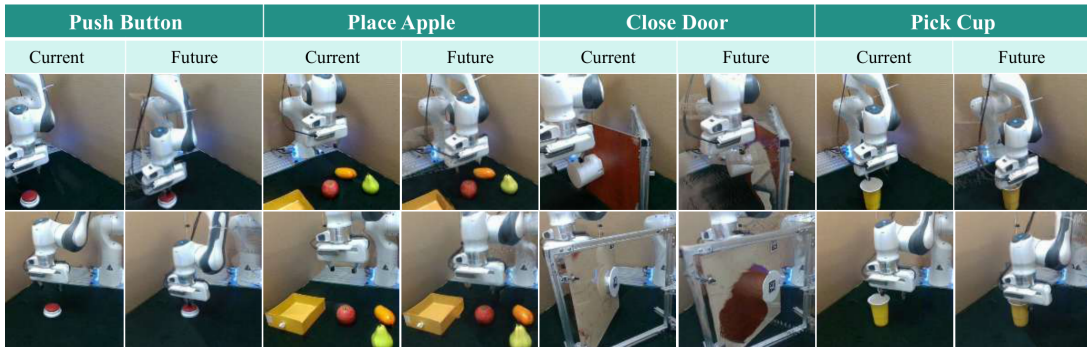
Ablation of the denoising framework. Executing the init actions directly, without refinement, fails: reconstruction errors under partial occlusion cause misaligned contacts, and the arm keeps pushing forward without reaching the correct position. Refinement is therefore essential for physically consistent interaction.

Table 3.7: Average novel-view synthesis quality for current and future scenes (three RLBench tasks) [2]

Method / state	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
ManiGaussian / current	16.38	0.396	0.688
ManiGaussian / future	15.77	0.374	0.721
GAF / current	27.92	0.782	0.131
GAF / future	26.30	0.760	0.145

Spatial generalization and multi-task learning. When objects are placed on a grid covering the entire workspace, Diffusion Policy fails near the boundaries and corners, while GAF succeeds even at the boundaries and is less sensitive to corners. When a single network is trained jointly on four tasks, GAF’s average success drops by only 11.25 points (to 70% on these tasks), compared with a 21-point drop for Diffusion Policy (to 40.5%), indicating that the 4D representation scales better to multiple tasks.

Real-world deployment. On a Franka Emika Panda arm with three calibrated Realsense D435i cameras (two fixed and one on the wrist) and 20 demonstrations per task, GAF achieves 10/10 on *push button*, 8/10 on *close door*, 7/10 on *open door*, 7/10 on *pick cup*, and 6/10 on *place apple*. Figure 3.2.5 shows current and predicted future renderings. Failures concentrate on grasping the apple or the door handle and are attributed to the lack of force feedback, suggesting tactile sensing or richer gripper-state data as remedies.

**Figure 3.2.5:** Real-world GAF results: renderings from current and predicted future Gaussians on four tasks [2]

3.2.8 Discussion

By embedding dynamics directly into an explicit scene representation, GAF closes the mismatch between static scene understanding and dynamic action generation. The pre-

dicted future Gaussians are not merely auxiliary supervision, as in prior Gaussian world models, but the very medium from which actions are extracted, producing a tight, interpretable coupling between prediction and control. The feed-forward design requires only sparse, unposed RGB views and runs in real time on a single GPU. Its principal limitation, acknowledged by the authors, is the absence of semantic or task-level understanding; incorporating language models to supply high-level priors is proposed as future work.

3.3 World Model-based Perception for Visual Legged Locomotion (WMP)

This paper presents a simple yet effective end-to-end framework that combines Model-Based Reinforcement Learning (MBRL) world modeling with sim-to-real transfer for vision-based quadrupedal locomotion [3]. Instead of the two-stage privileged teacher-student paradigm, WMP builds a Recurrent State-Space Model (RSSM) of the environment in simulation and learns a policy directly on top of the world model’s recurrent state, thereby avoiding both the information gap between teacher and student and the labor-intensive design of privileged observations.

3.3.1 Problem Statement and Motivation

Reinforcement learning in simulation followed by sim-to-real transfer has produced robust legged locomotion. Blind policies that use only proprioception can walk over slopes and stairs, but terrains such as gaps and pits must be perceived *in advance*, which requires vision. Learning directly from dense depth pixels with RL is extremely data-inefficient, and because a forward-facing camera cannot see beneath the robot, the policy must also remember past observations. The dominant solution is **privileged learning**: a teacher policy is trained with low-dimensional privileged information such as *scandots* (terrain heights sampled around the robot), and a vision-based student policy is then trained to imitate it.

Privileged learning has two limitations. First, the student cannot perfectly recover the teacher’s behaviour, and the gap grows when the privileged information and images carry different information. Second, privileged information must be designed by hand for

each terrain. As Figure 3.3.1 shows, sparse scandots cannot resolve the precise distance to the walls of a narrow *Tilt* passage and cannot represent off-ground obstacles such as the bar of a *Crawl* obstacle, whereas depth images represent both. Inspired by the observation that animals learn to traverse terrain through an internal model of the world rather than privileged knowledge, WMP asks whether a learned world model can serve as the perception module.

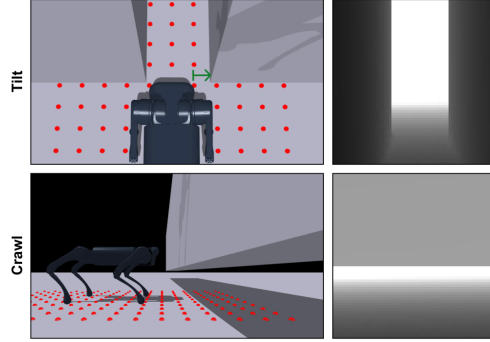


Figure 3.3.1: Scandots (left) and corresponding depth images (right) for *Tilt* and *Crawl* terrains: scandots cannot distinguish precise boundaries or represent off-ground obstacles [3]

3.3.2 Problem Formulation

Locomotion is formulated as the POMDP of Equation (2.1). In vision-based locomotion the observation consists of proprioception o_t^p and a depth image d_t , while the underlying state additionally contains privileged information s_t^{pri} available only in simulation:

$$o_t := (o_t^p, d_t), \quad s_t := (o_t, s_t^{\text{pri}}). \quad (3.16)$$

The proprioception $o_t^p \in \mathbb{R}^{45}$ contains base angular velocity, projected gravity, velocity commands, joint positions and velocities, and the previous action. The depth image $d_t \in \mathbb{R}^{64 \times 64}$ has a $58^\circ \times 58^\circ$ field of view. The privileged information contains scandots, foot contact forces, and randomized physical parameters. The action $a_t \in \mathbb{R}^{12}$ specifies joint position targets $q_d = q_{\text{stand}} + a_t$, tracked by a PD controller:

$$\tau = K_p (q_d - q) + K_d (\dot{q}_d - \dot{q}), \quad (3.17)$$

with $\dot{q}_d = 0$ and $(K_p, K_d) = (40, 1.0)$ on the real robot.

3.3.3 World Model Learning

WMP adopts an RSSM variant (Section 2.4) with one important modification: because rendering depth images in simulation and running the RSSM on board are both expensive, the world model runs at a *lower frequency* than the policy and updates its recurrent state only every k time steps (Figure 3.3.2). The four components are

$$\text{Recurrent model: } h_t = f_\phi(h_{t-k}, z_{t-k}, a_{t-k:t-1}), \quad (3.18)$$

$$\text{Encoder: } z_t \sim q_\phi(\cdot \mid h_t, o_t), \quad (3.19)$$

$$\text{Dynamic predictor: } \hat{z}_t \sim p_\phi(\cdot \mid h_t), \quad (3.20)$$

$$\text{Decoder: } \hat{o}_t \sim p_\phi(\cdot \mid h_t, z_t). \quad (3.21)$$

The recurrent model is a GRU; the encoder and decoder use CNNs for the depth image and MLPs for proprioception. Note that the recurrent update consumes the whole action sequence $a_{t-k:t-1}$ executed since the last update. Following DreamerV3, the components are trained jointly on trajectory segments of length L with the loss

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi} \left[\sum_{t=n \cdot k}^L -\ln p_\phi(o_t \mid z_t, h_t) + \beta \text{KL}(q_\phi(\cdot \mid h_t, o_t) \parallel p_\phi(\cdot \mid h_t)) \right], \quad (3.22)$$

where n is a non-negative integer. Predicting proprioception as well as depth encourages the model to capture physical properties of the environment.

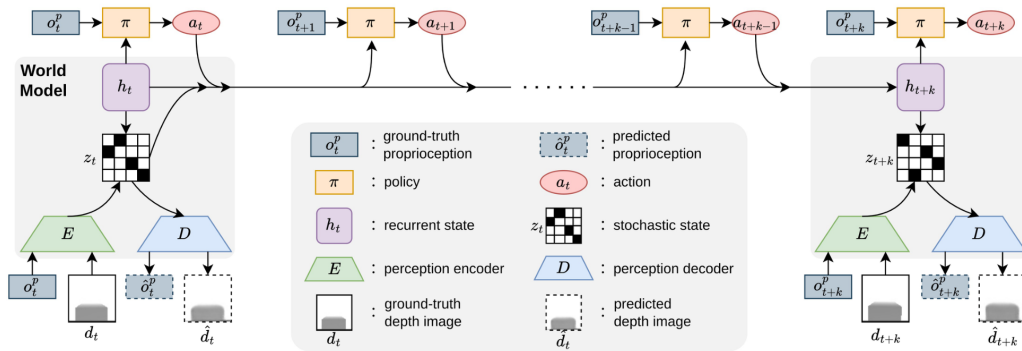


Figure 3.3.2: The WMP framework: the RSSM world model runs every k steps and its recurrent state feeds the policy [3]

3.3.4 Policy Learning

The key insight of WMP is that the recurrent state h_t of a well-trained world model contains enough information to predict the future and is therefore akin to the underlying Markovian state. The policy takes h_t as input together with the current proprioception, for every step between two world-model updates:

$$a_{t+i} \sim \pi_\theta(\cdot \mid o_{t+i}^p, \text{sg}(h_t)), \quad \forall i \in [0, k-1], \quad (3.23)$$

where $\text{sg}(\cdot)$ is the stop-gradient operator, so policy gradients do not alter the world model. An **asymmetric actor–critic** is used, in which the critic additionally observes the privileged information:

$$v(s_{t+i}) \sim V_\theta(\cdot \mid o_{t+i}^p, \text{sg}(h_t), s_{t+i}^{\text{pri}}). \quad (3.24)$$

The authors find that h_t is essential even for the critic, because scandots cannot represent Tilt and Crawl terrains. Actor and critic are trained with PPO on simulator data. Unlike Dreamer-style MBRL, WMP does *not* train the policy on imagined rollouts: since the world model is itself trained on simulator data, it cannot be more accurate than the simulator, and sampling from the parallel simulator is already cheap.

3.3.5 Training Environment and Rewards

Training uses the `legged_gym` codebase on the Isaac Gym simulator with 4096 parallel Unitree A1 robots on six terrain types—Slope, Stair, Gap, Climb, Tilt, and Crawl—with increasing difficulty levels and the terrain curriculum of Rudin et al.: a robot is promoted when it crosses its terrain and demoted when it travels less than half the commanded distance. The policy acts at 50 Hz (0.02 s per step); depth images are computed every k steps and delivered with a 100 ms latency to aid transfer, and physical parameters are randomized.

The robot tracks a command $(v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}})$. Instead of hand-selected waypoints, a simple clipped velocity-tracking reward is used,

$$r_{\text{tracking}} = \exp\left(-\left(\min(v_{xy}, v_{xy}^{\text{cmd}} + 0.1) - v_{xy}^{\text{cmd}}\right)^2 / \sigma\right), \quad (3.25)$$

which lets the robot exceed the commanded speed when necessary, for example to jump a gap. Penalties discourage getting stuck or turning around obstacles, and an Adversarial Motion Priors (AMP) style reward encourages natural gaits:

$$r_{\text{style}}(s, s') = \max[0, 1 - 0.25 (D_\psi(s, s') - 1)^2], \quad (3.26)$$

where the discriminator D_ψ is trained with a least-squares objective and gradient penalty to distinguish state transitions from a reference motion dataset from those produced by the policy.

3.3.6 Simulation Results

WMP is compared with a **Teacher** (oracle with scandots), a **Student** reproduced from Cheng et al. (ConvNet-RNN imitating the teacher from depth), a **Blind** ablation without depth, and **WMP w/o Prop** without proprioception in the world model. Table 3.8 and Figure 3.3.3 report returns and velocity tracking errors averaged over 100 trajectories. WMP attains near-teacher returns on all terrains where the teacher is applicable (e.g. 32.37 vs. 32.80 on Gap) while the student falls far behind (27.07). Removing depth collapses performance on every terrain except Slope, and removing proprioception degrades performance slightly. Teacher and Student are not applicable to Tilt and Crawl because scandots cannot describe these terrains.

Table 3.8: Simulation comparison of WMP (return $\uparrow$ / velocity tracking error $\downarrow$), mean over 100 trajectories [3]

	Method	Slope	Stair	Gap	Climb	Tilt	Crawl
Return	WMP (ours)	36.55	35.06	32.37	34.64	34.73	36.60
	Teacher (oracle)	36.70	35.07	32.80	35.32	N/A	N/A
	Student	36.31	34.93	27.07	33.42	N/A	N/A
	Blind	33.96	23.56	9.80	14.11	3.94	8.92
	WMP w/o Prop	36.07	34.62	30.73	34.67	30.78	36.41
Tracking error	WMP (ours)	0.008	0.013	0.26	0.13	0.02	0.006
	Teacher (oracle)	0.007	0.012	0.23	0.09	N/A	N/A
	Student	0.008	0.015	0.35	0.16	N/A	N/A
	Blind	0.019	0.033	0.33	0.26	0.08	0.051
	WMP w/o Prop	0.009	0.017	0.28	0.14	0.08	0.013

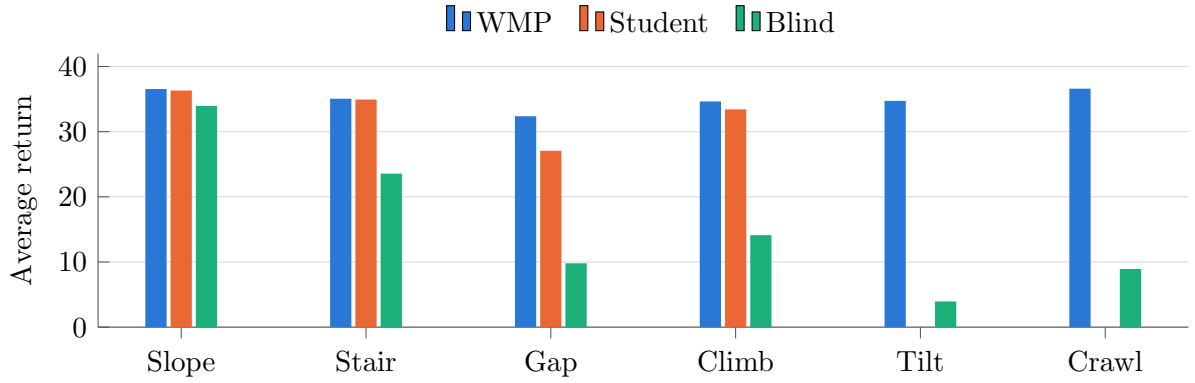


Figure 3.3.3: Simulated returns of WMP, the privileged-learning Student, and the Blind ablation on six terrains, plotted from [3]. The Student is not applicable to Tilt and Crawl (no bar).

3.3.7 Empirical Analysis of the World Model

Recurrent state visualization. A t-SNE projection of h_t collected on the six terrains (Figure 3.3.4, left) shows clearly separated clusters, with only slight overlap between Slope and Climb, whose depth images are similar (a climb is essentially a 90° slope). The recurrent state therefore encodes terrain type without any explicit supervision.

Model interval and training length. Varying k from 2 to 30 shows that smaller intervals yield higher simulated returns because the robot reacts faster (Figure 3.3.4, right). On the real A1, however, acquiring a depth image and running the world model takes about 40 ms, so $k = 5$ (0.1 s) is chosen as a trade-off. Training segments of about one second already give acceptable performance—intuitively, what the robot saw a second ago is roughly under its feet—while very long segments make back-propagation through the RSSM inefficient; 6.4 s segments are used in the main experiments.

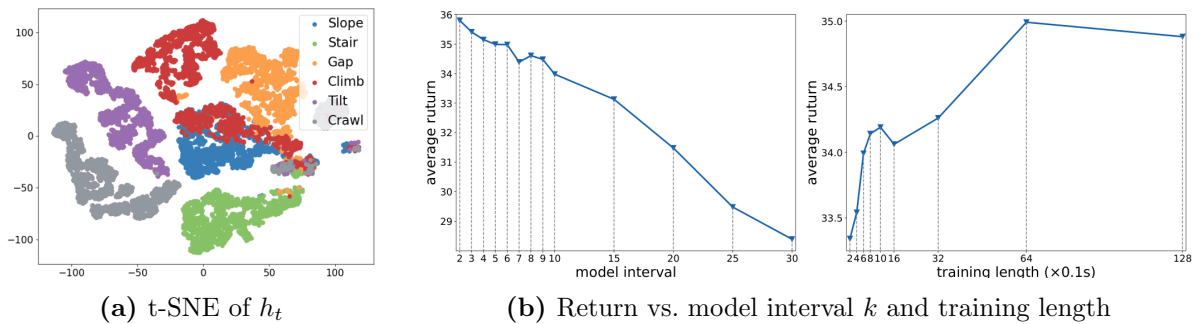


Figure 3.3.4: Empirical study of the WMP world model [3]

Real-world prediction. Given only the initial observation and the executed action sequence, the world model trained purely in simulation predicts real depth sequences accurately, especially at the critical location the robot is about to traverse (Figure 3.3.5). For the Crawl obstacle the predicted shape differs from reality because such shapes never appeared in simulation, yet the position and angle of the traversable crevice are consistent—evidence that latent-space world modelling generalizes and helps explain WMP’s smooth sim-to-real transfer.

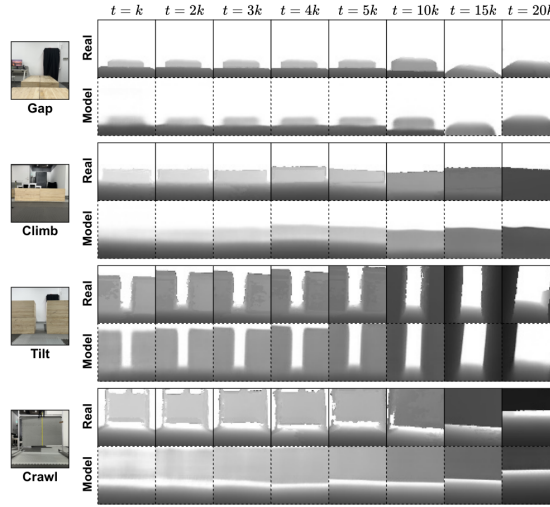


Figure 3.3.5: Real-world depth images and long-horizon predictions of the WMP world model on Gap, Climb, Tilt, and Crawl [3]

3.3.8 Real-World Evaluation

All policies run on the A1’s onboard Jetson NX. Depth from the front RealSense D435i is read at 60 Hz (424×240), filtered spatially and temporally to reduce noise, cropped and down-sampled to 64×64 , and fed to the world model with 100 ms latency. Five terrains of increasing difficulty are tested over ten trials each (Figure 3.3.6). The Student succeeds on easy Stair, Gap, and Climb settings but fails at harder levels and cannot handle Tilt or Crawl at all. WMP traverses a Gap of 85 cm (about $2.1 \times$ body length), a Climb of 55 cm (about $2.2 \times$ robot height), and a Crawl clearance of 22 cm (about $0.8 \times$ robot height), close to the hardest simulated levels and the best reported traversal performance on the A1. Ablating proprioception or images degrades performance. Outdoor deployment in a park shows consistent behaviour: the robot climbs stairs, mounts platforms up to 45 cm, and traverses grass and gravel.

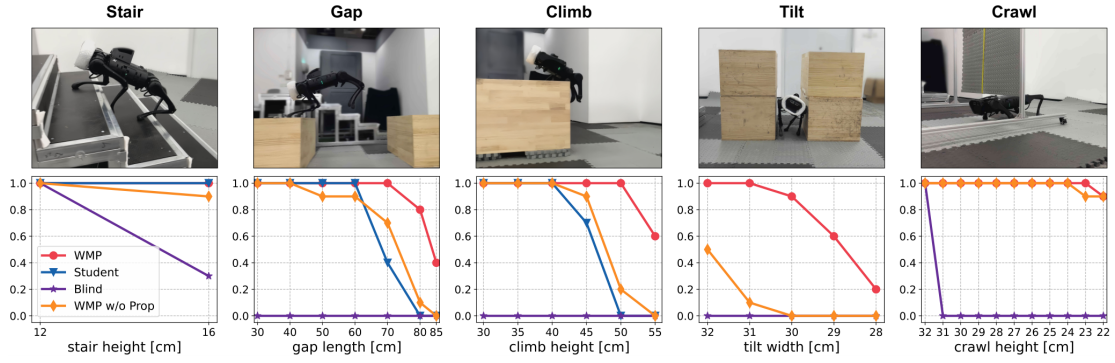


Figure 3.3.6: Real-world success rates of WMP, Student, Blind, and WMP w/o Prop over ten trials per difficulty level [3]

3.3.9 Discussion

WMP demonstrates that a world model trained entirely in simulation can serve as a transferable perceptual front-end for real robots: the recurrent state compresses a history of high-dimensional depth images into an informative, terrain-discriminative representation. This single-stage alternative to privileged learning removes teacher–student performance gaps and generalizes across terrains that hand-designed privileged observations cannot represent. Its limitations are that the model cannot exceed the simulator’s fidelity, that hardware latency forces a coarse model interval, and that long training segments are inefficient to back-propagate. The authors suggest training the world model on a mixture of simulated and real data and adding modalities such as touch.

3.4 WMNav: Integrating Vision-Language Models into World Models for Object Goal Navigation

This study proposes a fully modular, training-free world-model framework for Zero-Shot Object Navigation (ZSON), in which Vision-Language Models (VLMs) carry out all perception, prediction, planning, and reasoning [4]. Instead of interacting with the environment to discover the target by trial and error, the agent imagines the likely outcome of each direction of travel—exploiting the commonsense knowledge of indoor layouts embedded in VLMs—and stores these predictions in an online memory that steers exploration.

3.4.1 Problem Statement and Motivation

In object goal navigation, an agent placed in an unseen indoor environment must find and stop near an instance of a given category (e.g. *bed*, *sofa*, *toilet*). In the zero-shot setting the categories and environments are not seen during training. Existing methods are either *network-based*, training policies with RL or IL on large datasets, or *map-based*, building semantic maps and choosing frontiers or waypoints; the latter depend on accurate, complex map construction and do not fully exploit VLMs trained on egocentric images. Most importantly, both kinds of methods must actually interact with the environment to understand it, rather than anticipating the outcomes of their choices. VLFM builds a value map of target likelihood using BLIP-2, but its limited reasoning makes complex planning difficult. Humans, by contrast, can imagine a room layout from visual cues and common design patterns—walking along a corridor usually leads to new rooms, and searching a bathroom for a sofa is unwise. WMNav hypothesizes that VLMs, trained on vast amounts of first-person imagery, have acquired enough commonsense to play the role of such a world model (Figure 3.4.1).

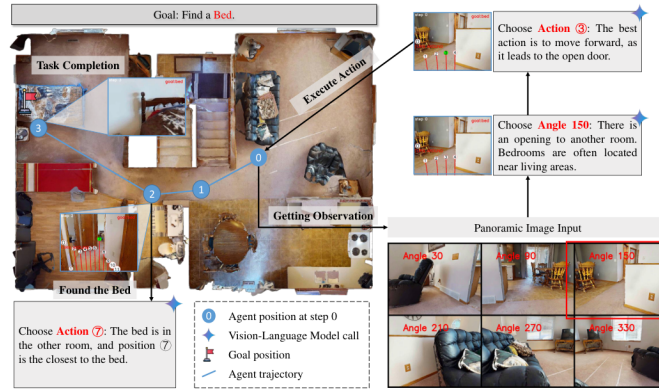


Figure 3.4.1: World model navigation with a VLM: the model estimates the goal’s likelihood in each panoramic direction, plans an intermediate subtask, and selects an action [4]

3.4.2 Task Definition and Framework Overview

At each step t the agent receives an RGB-D observation O_t and its pose P_t and chooses an action. Instead of the usual discrete action set $\{\text{Stop, Move Forward, Turn Left, Turn Right, Look Up, Look Down}\}$, WMNav uses polar-coordinate actions $a_t = (r_t, \theta_t)$ specifying a direction and a distance. An episode succeeds if the agent stops within a

threshold distance d_{thres} of a goal instance.

The framework (Figure 3.4.2) begins by rotating the agent through the six headings $\mathcal{A} = \{30^\circ, 90^\circ, \dots, 330^\circ\}$ (in steps of 60°) to capture six RGB-D images forming a panorama I_t^{pan} . The **world model** comprises a PredictVLM together with a memory built from the Curiosity Value Map and a cost; it receives no reward from the environment and is used only to predict and simplify the future state of the world. The **navigation policy** comprises a PlanVLM and a ReasonVLM, whose prompts are configured by the cost, so that the policy is optimized through prompting without fine-tuning any VLM.

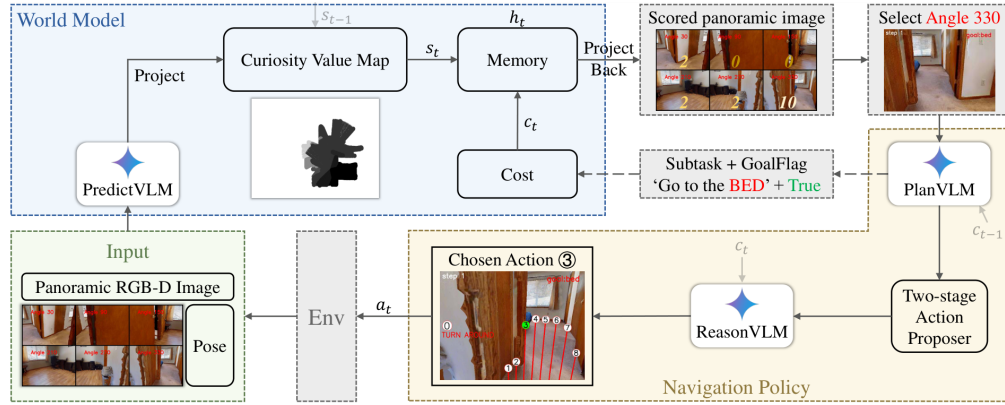


Figure 3.4.2: The WMNav framework with VLM-based world model and navigation policy [4]

3.4.3 World Model: Prediction and Curiosity Value Map

VLM-based state prediction. The panoramic image is given to PredictVLM with a structured prompt asking it to score every direction from 0 to 10: a direction with no feasible path and no target scores 0, a direction in which the goal is visible scores 10, and intermediate cases receive intermediate scores, each with a textual explanation returned in JSON format:

$$\text{Score}_t = \text{PredictVLM}(I_t^{\text{pan}}). \quad (3.27)$$

Curiosity Value Map construction. The Curiosity Value Map M^{cv} is a top-down grid in which each cell stores the curiosity value of that location (0–10). Cells are initialized to 10, since at the start the target could be anywhere. At each step the scores are projected from the egocentric navigable areas onto a top-down map M_t^{nav} using depth D_t and pose

P_t , and fused with the previous map by a pixel-wise minimum:

$$M_t^{nav} = \text{Projection}(\text{Score}_t), \quad M_t^{cv}(u, v) = \min(M_{t-1}^{cv}(u, v), M_t^{nav}(u, v)). \quad (3.28)$$

The minimum ensures that regions already observed to lack the goal (score 0) stay suppressed, preventing redundant back-and-forth exploration (Figure 3.4.3).

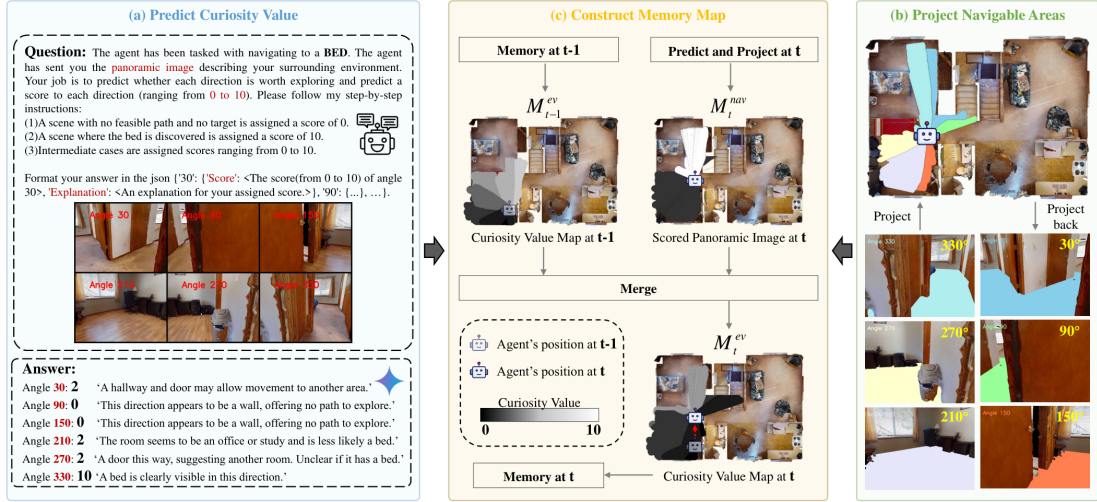


Figure 3.4.3: (a) Predicting curiosity values for each panoramic direction; (b) projection of navigable areas between egocentric and top-down views; (c) fusing predictions into the Curiosity Value Map [4]

Cost. Because a VLM world model receives no environmental reward, WMNav defines a *cost* consisting of the current subtask and a goal flag indicating whether the goal has been seen. The cost is inserted into the prompts of PlanVLM and ReasonVLM, implicitly optimizing the policy’s outputs. The memory h_t therefore consists of the map s_t and the cost c_t .

3.4.4 Subtask Decomposition

Planning directly towards an unseen goal provides very sparse feedback. WMNav therefore decomposes the goal into intermediate subtasks, as a human would: find the corridor to the bedroom, reach the bedroom door, then approach the bed. After the map update, curiosity values are projected back onto the current navigable areas, the average score of each direction is computed, and the direction α with the highest score is selected.

Its image $I_t(\alpha)$, the explanation for choosing it, and the previous subtask are given to PlanVLM, which outputs a new subtask and goal flag:

$$\text{SubTask}_t, \text{GoalFlag}_t = \text{PlanVLM}(I_t(\alpha), \text{SubTask}_{t-1}). \quad (3.29)$$

Comparing the plan with the new observation provides feedback that mitigates hallucination: if the previous subtask is not complete, the agent continues it; if it is complete, a new one is chosen.

3.4.5 Two-Stage Action Proposer

Candidate actions are generated following VLMNav: K polar vectors are sampled at regular angular intervals $\Delta\theta$ within the navigable area of $I_t(\alpha)$, giving $A_t^{\text{init}} = \{(r_{t,i}, \theta_{t,i})\}_{i=1}^K$. Actions leading into already-explored regions are filtered out using an exploration state map, distances and angular spacing are constrained, and the remaining K' candidates A_t^{cand} are drawn as numbered arrows on the image to produce I_t^{ann} (Figure 3.4.4). Decision making is split into two stages:

- **Exploration stage:** ReasonVLM, configured as ActionVLM, chooses the candidate that best serves the current subtask:

$$a_t = \text{ReasonVLM}(\text{SubTask}_t, \text{Goal}_t, I_t^{\text{ann}}), \quad a_t = (r_t, \theta_t). \quad (3.30)$$

- **Goal-approaching stage:** once the goal is detected, the length constraint is removed and sampling becomes denser so that some vector ends at the goal's ground location; ReasonVLM, configured as GoalVLM, selects it. Rather than trusting the VLM to decide when to stop, stopping is triggered geometrically:

$$\text{StopFlag} = \begin{cases} \text{True}, & \text{DistanceToGoal} < d_{\text{thres}} \\ \text{False}, & \text{otherwise.} \end{cases} \quad (3.31)$$

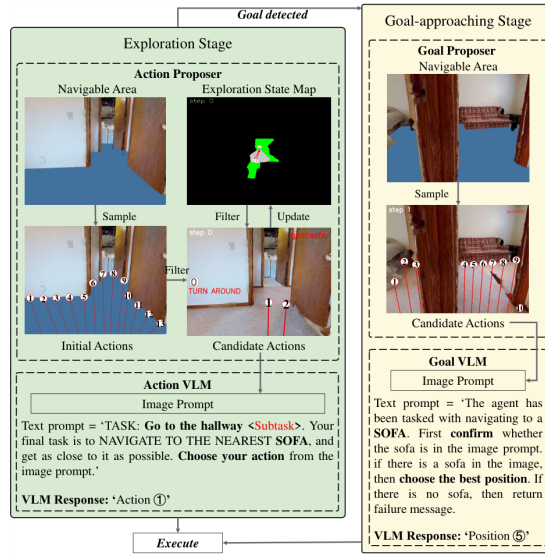


Figure 3.4.4: Reasoning the action: the exploration stage filters sampled actions for Action-VLM, and the goal-approaching stage densely samples positions for GoalVLM [4]

3.4.6 Experimental Setup

WMNav is evaluated in the Habitat simulator on three benchmarks: **HM3D v0.1** (2000 validation episodes in 20 environments, 6 goal categories; used in the Habitat 2022 ObjectNav challenge), **HM3D v0.2** (1000 validation episodes with improved geometry and semantic labels), and **MP3D** (11 scenes, 2195 validation episodes, 21 goal categories). Metrics are SR and SPL (Equation 2.18). Table 3.9 lists the agent configuration; Gemini VLMs are used by default for their low cost and high effectiveness.

Table 3.9: Agent configuration used in WMNav experiments [4]

Parameter	Value
Maximum navigation steps	40
Agent body	Cylinder, radius 0.18 m, height 0.88 m
Camera	Egocentric RGB-D, 640×480 , horizontal FoV 79° , pitched down 14°
Panorama headings	$30^\circ, 90^\circ, 150^\circ, 210^\circ, 270^\circ, 330^\circ$
Success threshold d_{thres}	1.0 m
VLMs	Gemini series (default); Qwen2.5-VL in ablations
Training	None (training-free, zero-shot; no detectors or pre-built maps)

3.4.7 Results and Analysis

Comparison with the state of the art. Table 3.10 compares WMNav with supervised and zero-shot methods. WMNav outperforms all zero-shot methods, improving SR by 3.2 points and SPL by 3.2 points on HM3D and SR by 13.5 points and SPL by 1.1 points on MP3D over the previous best. Even including supervised methods, it achieves the best SR on MP3D and the best SPL on HM3D. The authors argue that LLM-based methods such as ESC lose spatial detail when converting images to text; detector-based semantic maps (OpenFMNav) are complex and under-use the VLM; top-view inputs (TopV-Nav) do not exploit VLMs’ egocentric training; whereas the simple Curiosity Value Map directly exploits the VLM’s egocentric scene understanding.

Table 3.10: Object navigation results on HM3D v0.1 and MP3D. TF: training-free; ZS: zero-shot [4]

Method	TF	ZS	HM3D		MP3D	
			SR (%)	SPL (%)	SR (%)	SPL (%)
Habitat-Web	×	×	41.5	16.0	31.6	8.5
OVRL	×	×	–	–	28.6	7.4
OVRL-V2	×	×	64.7	28.1	–	–
ZSON	×	✓	25.5	12.6	15.3	4.8
PSL	×	✓	42.4	19.2	18.9	6.4
PixNav	×	✓	37.9	20.5	–	–
SGM	×	✓	60.2	30.8	37.7	14.7
VLFM	×	✓	52.5	30.4	36.4	17.5
CoW	✓	✓	–	–	9.2	4.9
ESC	✓	✓	39.2	22.3	28.7	14.2
L3MVN	✓	✓	50.4	23.1	–	–
VoroNav	✓	✓	42.0	26.0	–	–
OpenFMNav	✓	✓	54.9	24.4	–	–
TopV-Nav	✓	✓	45.9	28.0	31.9	16.1
WMNav (ours)	✓	✓	58.1	31.2	45.4	17.2

Ablation of modules and memory. Table 3.11 shows ablations on HM3D v0.2. Adding subtask decomposition (a→b), the Curiosity Value Map (a→d), and the two-stage action proposer (e→f) each improves SR, reaching 72.2% with all modules. Among

memory strategies, a *text-image* memory, in which the VLM describes observations in text and plans over a top-down trajectory map, performs even worse than no memory, because feeding the VLM text-image combinations easily induces hallucinations. The quantitative Curiosity Value Map forces rigorous outputs and gives the best SR and SPL.

Table 3.11: Ablation of modules and memory strategies on HM3D v0.2. SD: subtask decomposition; TAP: two-stage action proposer; CVM: Curiosity Value Map [4]

	Memory	SD	TAP	SR (%)	SPL (%)
a	No memory	×	×	65.8	25.8
b	No memory	✓	×	67.4	33.1
c	Text-image	✓	×	62.0	29.6
d	CVM	×	×	69.5	34.9
e	CVM	✓	×	70.1	34.7
f	CVM	✓	✓	72.2	33.3

Effect of the VLM. When every VLM in the framework is replaced by the same model (Figure 3.4.5), performance increases with model scale—Qwen2.5-VL-3B reaches 29.7% SR and the 7B model 46.1%—and proprietary Gemini models perform best, with Gemini 1.5 Pro reaching 58.1% SR and 31.2% SPL. Even Gemini 1.5 Flash (53.5% SR) remains competitive with prior methods.

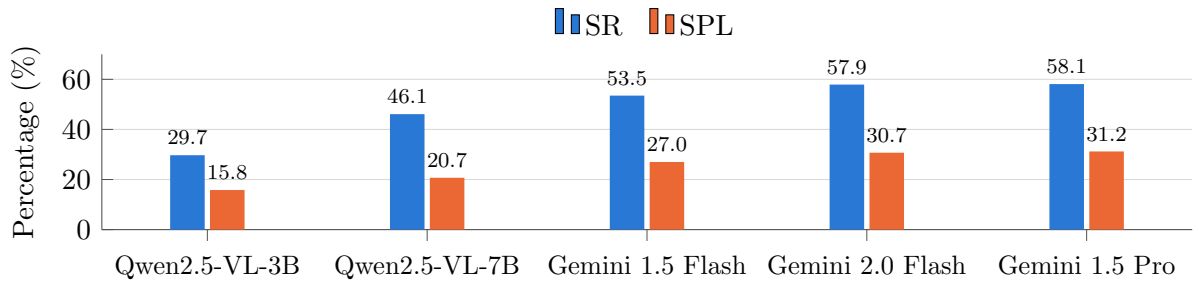


Figure 3.4.5: WMNav performance on HM3D v0.1 with different VLMs, plotted from [4]

3.4.8 Discussion

WMNav shows that pre-trained VLMs can act as abstract world models: prediction replaces costly exploration and curiosity scoring enforces reliable outputs. However, it relies on repeated calls to large, often proprietary VLMs and has been tested only in simulation.

Chapter 4

Discussion & Comparison

This chapter presents a detailed comparative analysis of the four reviewed research papers that instantiate world models across the manipulation, locomotion, and navigation domains of embodied AI. These studies collectively demonstrate how predictive internal models—whether explicit physics simulations, motion-augmented Gaussian fields, latent recurrent dynamics, or VLM-based imagination—are transforming robotics from reactive perception-action mappings into anticipatory, plan-before-act intelligence. While each paper shares the common goal of predicting the future state of the world to improve decision-making, they differ fundamentally in representation, learning signals, planning mechanisms, and deployment platforms. The following sections critically evaluate their contributions, advantages, and limitations, offering insights into their comparative strengths and areas for further improvement.

4.1 Analytical Focus

Each paper addresses a distinct challenge in world-model-driven embodied AI. Paper [1] tackles **precise, goal-conditioned manipulation of unseen articulated objects**, turning perception into search inside a physics simulator via an explicit digital twin. Paper [2] addresses the **mismatch between static scene understanding and dynamic action generation** by embedding motion into an explicit Gaussian scene representation, so actions are read directly from predicted geometry. Paper [3] targets **perception under partial observability** in legged locomotion, compressing depth-image and proprioceptive histories into a latent state that replaces hand-designed privileged information. Paper [4] explores **prediction without any environment-specific training**, delegating world modeling to pre-trained VLMs’ commonsense knowledge for zero-shot navigation. Together, these studies span physics-grounded, geometry-grounded, latent, and semantic

designs, united by imagining outcomes before acting.

4.2 Comparison of Methodologies

Table 4.1 summarizes the methodology, advantages, and limitations of each work. The table highlights that the four systems occupy complementary positions: none dominates on every axis, and each limitation is, to a large extent, the mirror image of the corresponding advantage.

Table 4.1: Comparison of world-model approaches for embodied AI

Paper	Methodology	Advantages	Limitations
DexSim2Real ² [1]: Explicit world model for articulated object dexterous manipulation	Interactive perception + 3D AIGC digital twin construction; sampling-based MPC (iCEM) with eigengrasp dimensionality reduction for dexterous hands.	Demonstration-free and RL-free; high precision (avg. error $\approx 7^\circ$); supports multiple end effectors and unseen strategies such as tool use.	Requires observable state changes during interaction; 3D AIGC meshes may misalign for novel objects; minutes-scale trajectory search.
GAF [2]: Gaussian Action Field as a 4D representation for dynamic world modeling	Extends 3DGS with learnable per-Gaussian motion; ViT-based feed-forward reconstruction; ICP-derived init action refined by action-vision-aligned diffusion.	Unifies reconstruction, prediction, and action; needs only sparse unposed RGB; +7.3% success over the best baseline and real-time inference (<0.3 s).	Lacks semantic/task-level understanding; partial occlusions cause misaligned contacts; needs some real action data for refinement.

Paper	Methodology	Advantages	Limitations
WMP [3]: World Model-based Perception for visual legged locomotion	RSSM world model trained in simulation; policy on recurrent latent state via asymmetric actor-critic PPO; AMP-style natural-gait rewards.	Removes privileged teacher-student gap; near-teacher returns; best real-world traversal on Unitree A1 (Gap 85 cm, Climb 55 cm, Crawl 22 cm).	World model bounded by simulator fidelity; latency forces coarse model interval ($k = 5$); no real-data model training yet.
WMNav [4]: Integrating Vision-Language Models into world models for object goal navigation	PredictVLM scores directions into an online Curiosity Value Map; sub-task decomposition; two-stage action proposer; all reasoning by prompted VLMs.	Zero-shot and training-free; +3.2%/+13.5% SR over the best zero-shot methods on HM3D/MP3D; reduces risky exploration through prediction.	Dependent on proprietary VLM capability and cost; susceptible to hallucination; indoor object-goal setting only.

4.3 Representation, Learning Signal, and Control Strategy

A finer-grained comparison of how each world model is represented, built, and used is given in Table 4.2. Two observations stand out. First, the *construction cost* of the world model is inversely related to its *generality*: DexSim2Real² builds a new, highly accurate model for every object instance; GAF and WMP learn a model per task family; and WMNav uses a single frozen model for every environment and category. Second, the *control strategy* follows from the representation: an explicit simulator invites search (MPC), a geometric field invites registration (ICP) followed by learned refinement, a latent state invites a learned policy, and a semantic model invites prompting.

Table 4.2: World-model representation, construction, and use in the four reviewed works

Aspect	DexSim2Real ²	GAF	WMP	WMNav
World state	URDF digital twin (meshes + joints)	3D Gaussians $\{\mu, f\}$	RSSM state (h_t, z_t)	Curiosity Value Map + subtask cost
Transition model	Physics engine (SAPIEN)	Learned displacement $\Delta\mu$	GRU recurrent model	VLM imagination (PredictVLM)
How it is obtained	K interactions + 3D AIGC + kinematic estimation	Trained on RGB video frames	Trained on simulated rollouts	Pre-trained VLM, prompted
Supervision	None for the model itself; affordance from sim or human video	Photometric (LPIPS + MSE); 20 demos/task for refinement	Reconstruction + KL; PPO rewards	None (training-free)
Decision making	iCEM sampling-based MPC	ICP init action + diffusion refinement	PPO policy on $sg(h_t)$	PlanVLM + ReasonVLM with action proposer
Action space	Joint increments (7–9 D with eigen-grasp)	End-effector poses + gripper state	12 joint targets at 50 Hz	Polar vectors (r, θ)
Decision latency	1.5–3 min per trajectory	<0.3 s per 8 actions	Real time on Jetson NX	Several VLM calls per step

4.4 Model Performance and Evaluation

All four studies provide rigorous quantitative evaluation on real robots or standard benchmarks.

- Paper [1] achieved average manipulation errors of 7.2° (laptop) and 6.8° (cabinet) with the suction gripper versus $16.0^\circ/15.3^\circ$ for UMPNet, and world-model construction accuracy with joint-axis angle error below 9° across four object categories.
- Paper [2] reported a 60.4% average success rate on nine RLBench tasks, exceeding Diffusion Policy by 15.7%, Act3D by 7.3%, and ManiGaussian by 10.3%, alongside +11.54 dB PSNR reconstruction gains and 6–10/10 success on five real Franka tasks.
- Paper [3] attained near-teacher returns in simulation and the best real-world traversal performance reported on a Unitree A1: Gap 85 cm, Climb 55 cm, Tilt 28 cm, and Crawl 22 cm, terrains on which the privileged student baseline partially or completely fails.
- Paper [4] delivered state-of-the-art zero-shot navigation with 58.1% SR / 31.2% SPL on HM3D and 45.4% SR / 17.2% SPL on MP3D, and 72.2% SR on HM3D v0.2 in

ablation configuration.

Despite these strong results, the evaluation settings differ substantially—single fixed-arm tabletop scenes, RLBench and Franka tasks, indoor quadruped courses, and Habitat photorealistic scans—so cross-paper numbers indicate domain-specific maturity rather than direct superiority of one representation over another. Table 4.3 summarizes the experimental platforms.

Table 4.3: Experimental platforms and evaluation protocols

Aspect	DexSim2Real ²	GAF	WMP	WMNav
Robot	ROKAE xMate3Pro 7- DOF arm	Franka Emika Panda	Unitree A1 quadruped	Simulated cylin- drical agent
End effector / body	Suction, Robotiq 2F-140, Allegro Hand, XHand	Panda Hand	12 actuated joints	Radius 0.18 m, height 0.88 m
Sensors	RealSense D415 + side RGB camera	3 RealSense D435i (2 fixed, 1 wrist)	Front RealSense D435i depth + proprioception	RGB-D 640 × 480, HFoV 79°
Simulator	SAPIEN	RLBench (Cop- peliaSim)	Isaac Gym (legged_gym)	Habitat
Benchmarks / objects	Drawers, cabinets, laptops, lamps	9 RLBench tasks + 5 real tasks	6 terrains, 5 real terrains	HM3D v0.1/v0.2, MP3D
Main metrics	$ \delta $, $ \delta_r $, RDE, IoU, axis error	Success rate, PSNR, SSIM, LPIPS	Return, tracking error, success rate	SR, SPL

4.5 Data and Environment Utilization

Data requirements diverge sharply across the four systems. Paper [1] needs only $K + 1$ real observation frames per object plus offline affordance training data, sourced either from simulated interaction (Where2Act on SAPIEN assets) or from large-scale egocentric human videos (VRB), the latter removing dependence on scarce interactable 3D assets. Paper [2] trains from RGB video frames alone for reconstruction—no depth, poses, or 3D ground truth—plus 20 demonstrations per RLBench task for action refinement. Paper [3] exploits massively parallel simulation (4096 robots in Isaac Gym) with terrain curriculum and domain randomization, requiring no real-world data at all. Paper [4] requires no training data whatsoever, relying entirely on frozen VLMs and the HM3D/MP3D evaluation scenes. This progression—from instance-specific reconstruction to zero-data prediction—

highlights how world models can trade data collection against prior knowledge, whether physical, geometric, or semantic.

4.6 Scalability and Deployment Potential

Deployment characteristics vary significantly across the proposed systems:

- Paper [1]’s pipeline runs trajectory search in about 1.5–3 minutes per task on a desktop CPU/GPU, suitable for deliberate household manipulation but not for reactive control; its modular design extends naturally to new end effectors and tools.
- Paper [2] operates feed-forward on a single GPU with sub-0.3 s action prediction and parallel prediction–execution threads, making it genuinely real-time and deployable on real manipulators.
- Paper [3] runs fully on-board a Jetson NX without external computation, with the model interval $k = 5$ chosen explicitly to absorb 40 ms of sensing and inference latency—evidence of careful hardware–algorithm co-design.
- Paper [4]’s reliance on cloud VLM APIs introduces latency and cost per step, restricting current deployment to non-time-critical navigation, though the framework improves automatically as VLMs advance.

In real-world implementation, the trade-off between model fidelity, planning latency, and computational footprint must be balanced against task tempo: manipulation tolerates minutes of deliberation, locomotion demands 50 Hz reactivity, and navigation lies in between.

4.7 Interpretability and Trustworthiness

World models differ markedly in how inspectable their predictions are. Paper [1] is the most interpretable: its digital twin is a human-readable URDF with explicit joint types, axes, and meshes, and planned trajectories can be visualized in simulation before execution. Paper [2] renders photorealistic images of both current and predicted future scenes, allowing direct visual verification of what the model believes will happen. Paper [4] produces natural-language explanations for every directional score and subtask, and its feedback loop between plan and observation explicitly counteracts VLM hallucination.

Paper [3], by contrast, encodes knowledge in a latent state; interpretability is indirect, through decoded depth predictions and t-SNE terrain clustering. This ordering suggests a tension between representation compactness and transparency that future world-model designs must negotiate.

4.8 A Design-Space View

The comparison can be condensed into a two-dimensional design space (Figure 4.8.1). The vertical axis represents *physical and geometric fidelity*—how precisely the model captures shapes, contacts, and dynamics—while the horizontal axis represents *semantic breadth and generality*—how many objects, categories, and environments the same model covers without retraining. DexSim2Real² sits at the high-fidelity, instance-specific corner; GAF trades a little physical grounding for a learned, task-family-level model; WMP captures terrain dynamics compactly but only for the terrain distribution it was trained on; and WMNav reaches the broadest semantic coverage with essentially no physical modelling. The empty upper-right region—models that are both physically precise and semantically general—is where future hybrid world models are expected to lie.

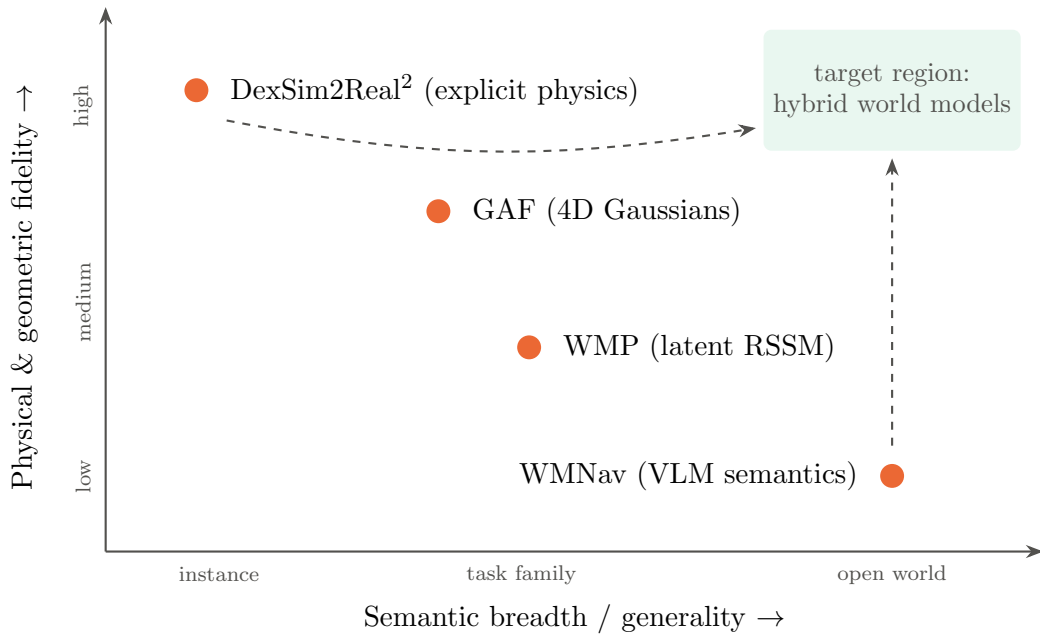


Figure 4.8.1: Qualitative placement of the reviewed world models in a fidelity–generality design space. Positions are a qualitative synthesis of the comparison in this chapter, not measured quantities.

4.9 Innovation and Research Contributions

Each paper introduces novel elements that advance the state-of-the-art:

- **Paper [1]** — Combines interactive perception, 3D AIGC digital-twin reconstruction of multi-part articulated objects, and eigengrasp-reduced sampling-based MPC for dexterous hands, with video-based affordance learning replacing scarce simulation assets.
- **Paper [2]** — Introduced the V-4D-A paradigm and the Gaussian Action Field, extracting actions from predicted Gaussian motion via ICP and refining them with actionable multiview RGB guidance.
- **Paper [3]** — First application of Dreamer-style RSSM world modeling to challenging vision-based legged locomotion with successful sim-to-real transfer, replacing the privileged learning paradigm.
- **Paper [4]** — Proposed a fully modular VLM world model for ZSON, with the online Curiosity Value Map as quantitative predictive memory and hallucination-mitigating subtask feedback.

Together, these contributions indicate a shift toward embodied agents that construct, maintain, and exploit predictive models of their surroundings as a first-class component of the control loop.

4.10 Practical Relevance and Real-World Application

From a deployment perspective, Paper [1] targets assistive and household robotics where precise operation of cabinets, drawers, laptops, and lamps matters, and its tool-use extension previews service robots that improvise with available implements. Paper [2]’s real-time pipeline suits industrial and tabletop manipulation cells where sparse cameras are cheaper than calibrated depth rigs. Paper [3] directly advances inspection, delivery, and search-and-rescue quadrupeds that must negotiate stairs, gaps, and crawl spaces outdoors, as demonstrated in park deployments. Paper [4] addresses domestic assistance—finding objects in unseen homes—without any environment-specific setup. Collectively, these models cover deliberate manipulation, agile locomotion, and semantic navigation,

demonstrating the versatility of the world-model paradigm across the full spectrum of embodied robotics.

4.11 Overall Comparative Insights

In synthesis, Paper [1] prioritizes precision through explicit physics, Paper [2] unification of prediction and action through 4D geometry, Paper [3] perceptual compression through latent dynamics, and Paper [4] zero-shot semantics through foundation models. Each contributes to a different dimension of world-model development:

- **Precision and Physical Grounding:** Papers [1]–[2] anchor predictions in explicit geometry and physics, enabling accurate contact-rich manipulation.
- **Sample Efficiency:** Papers [1] and [3] show that physics priors and simulated world-model training dramatically cut real-world data needs.
- **Real-Time Capability:** Papers [2]–[3] achieve on-board, low-latency inference; Paper [1] trades latency for precision; Paper [4] is bounded by VLM API speed.
- **Generality and Zero-Shot Ability:** Paper [4] generalizes to unseen scenes and categories without training, while Paper [1] generalizes to unseen actions such as tool use.

In conclusion, the four reviewed research papers collectively mark a significant evolution in embodied AI. The fusion of explicit digital twins, dynamic Gaussian representations, latent recurrent dynamics, and VLM-based imagination has transformed robots from reactive controllers into agents capable of predicting, planning, and explaining their behavior. Future research will likely converge these paradigms—combining the precision of explicit models, the real-time efficiency of feed-forward 4D representations, the compression of latent states, and the semantic breadth of foundation models—to create unified, resilient world models for the next generation of embodied intelligent systems.

Chapter 5

Applications

World-model-driven embodied agents are moving from laboratory demonstrations toward practical use. This chapter surveys application areas in which the capabilities demonstrated by the reviewed works—precise articulated manipulation, dynamic scene prediction, agile perceptive locomotion, and semantic navigation—are directly relevant. The discussion is grounded in the demonstrations reported by the papers and indicates which family of world model is most suitable for each setting.

5.1 Household and Assistive Robotics

Homes are full of articulated objects: kitchen cabinets, refrigerators, drawers, ovens, microwaves, laptops, and lamps. An assistive robot that helps elderly or physically impaired people must open and close these objects to specific configurations—a refrigerator door just wide enough to reach inside, a drawer opened half-way—rather than simply “open” or “closed”. DexSim2Real² addresses exactly this goal-conditioned setting and demonstrates it on drawers, cabinets, laptops, lamps, and a multi-part refrigerator. Its ability to build a model after a single interaction is important in homes, where every object instance is different and collecting demonstrations for each is infeasible. The tool-use capability further suggests robots that can reach an object at the back of a counter with a utensil, or pull a small drawer with a hook.

5.2 Industrial and Warehouse Manipulation

In factories and fulfilment centres, robots must handle varied parts, operate fixtures, and recover from disturbances in real time. GAF’s feed-forward pipeline, which infers actions from two unposed RGB views in under 0.3 s and generalizes across the workspace, suits such manipulation cells, where installing and calibrating dense depth sensors is costly.

The ability to render predicted future scenes also supports monitoring: a supervisor or an automated checker can compare the predicted outcome of an action with what actually happens and flag anomalies. Digital twins of the kind built by DexSim2Real² are likewise valuable for offline programming and virtual commissioning, in which robot motions are planned and verified in simulation before being deployed to the production line.

5.3 Inspection, Search-and-Rescue, and Field Robotics

Legged robots are increasingly deployed for industrial inspection (power plants, oil and gas facilities, construction sites), public safety, and search-and-rescue in collapsed or cluttered environments. These settings feature gaps, steps, narrow passages, and overhanging obstacles—exactly the Gap, Climb, Tilt, and Crawl terrains of WMP. Because WMP runs entirely on the robot’s onboard computer and does not require hand-designed privileged information for each obstacle type, it is well matched to field deployment where communication may be unreliable and obstacles are not known in advance. The park experiments, in which the robot climbed stairs, mounted platforms up to 45 cm, and walked on grass and gravel, illustrate this potential.

5.4 Domestic Service and Semantic Navigation

A service robot asked to “bring the remote from the sofa” or “check whether the stove is off” must first find the relevant object in a home it may never have mapped. Zero-shot object navigation, as addressed by WMNav, is a key capability for such robots. Because WMNav needs no training on the target environment and no pre-built maps, it could be deployed in a new home immediately, using commonsense priors (sofas are in living rooms, beds in bedrooms) to search efficiently. Similar reasoning applies to hospitality and retail robots that must locate items or locations in large indoor spaces, and to mobile manipulators that combine navigation with the manipulation capabilities of the other reviewed works.

5.5 Healthcare and Laboratory Automation

Hospitals and laboratories contain many articulated devices—sample cabinets, instrument doors, drawers—and require precise, safe, and verifiable operation. Explicit world

models are attractive here because every planned motion can be simulated and inspected before execution, providing a natural point for safety checks. Predictive models also help robots operate around people: by anticipating how a scene will evolve, a robot can avoid actions whose predicted outcome is unsafe.

5.6 Simulation, Testing, and Education

Beyond deployment, world models serve as tools for robot development. Digital twins reconstructed from a few observations can populate simulation environments with realistic articulated assets, alleviating the shortage of interactable 3D models noted by the DexSim2Real² authors. Latent world models that predict real-world observations, as in WMP, can be used to test policies offline before hardware trials. VLM-based world models can generate high-level plans and explanations that make robot behaviour easier to understand for students and non-expert users.

Table 5.1: Application domains and the most suitable world-model families

Application domain	Key requirement	Most relevant work
Household / assistive manipulation	Precise, goal-conditioned operation of unseen articulated objects	DexSim2Real ² [1]
Industrial manipulation cells	Real-time action inference from sparse, cheap cameras	GAF [2]
Inspection and search-and-rescue	On-board perceptive locomotion over unknown obstacles	WMP [3]
Domestic service navigation	Zero-shot search for objects in unmapped homes	WMNav [4]
Healthcare / laboratory automation	Verifiable, inspectable plans before execution	DexSim2Real ² [1], GAF [2]
Simulation and testing	Realistic assets and offline policy evaluation	DexSim2Real ² [1], WMP [3]

Chapter 6

Challenges and Future Directions

Despite significant progress in world-model-driven embodied AI, several technical, operational, and practical barriers still hinder effective deployment in unstructured real-world environments. This chapter first summarizes the challenges identified in each of the four reviewed studies, then discusses cross-cutting challenges common to all world models, and finally outlines promising research directions.

6.1 Challenges Identified in the Reviewed Works

6.1.1 DexSim2Real²: Explicit World Model for Dexterous Manipulation

The framework infers kinematic structure by analyzing inter-frame differences, so it strictly requires observable state changes of every movable part during interactive perception; parts that cannot be moved or seen cannot be modeled. For novel or uncommon objects, existing 3D AIGC techniques may generate meshes that are imperfectly aligned with real-world observations, degrading digital-twin accuracy and hence manipulation precision, since manipulation error correlates positively with joint position and orientation errors of the model. Real objects also exhibit unmodeled dynamics—elastic deformation of laptop lids and rail gaps that turn ideal prismatic joints into multi-DOF joints—which the rigid simulation cannot capture. Finally, trajectory search takes 1.5–3 minutes per task, precluding reactive behavior, and the current system is limited to a single object manipulated by one fixed arm.

6.1.2 GAF: 4D Gaussian World Modeling

Although GAF unifies reconstruction, prediction, and action, its purely geometric world model lacks semantic and task-level understanding, limiting context-aware manipulation. Reconstruction errors under partial occlusion lead to misaligned contacts during interac-

tion; without the diffusion-based refinement stage, executed init actions frequently fail, showing that visually plausible motion is not automatically physically feasible. The refinement stage reintroduces a dependence on real action demonstrations, and real-world failures concentrated in grasping (apple, door handle) reveal the absence of force feedback, suggesting the need for tactile sensing and richer gripper-state training data. Multi-task training also reduces performance on dissimilar tasks such as *lift lid up*.

6.1.3 WMP: World Model-based Perception for Locomotion

Because the RSSM is trained exclusively on simulated data, it cannot exceed the simulator’s fidelity; unmodeled real-world dynamics and never-seen obstacle shapes manifest as prediction discrepancies (e.g., the Crawl obstacle’s shape differs in predicted depth images). Real-world latency of depth acquisition and on-board inference (≈ 40 ms) forces a coarser world-model interval ($k = 5$), sacrificing the higher returns that faster model updates achieve in simulation. Backpropagating gradients through long RSSM training segments is inefficient, restricting usable history length to seconds. Training the world model with mixed simulated and real data, and incorporating additional sensing modalities such as touch, remain open future work.

6.1.4 WMNav: VLM World Models for Navigation

The framework inherits the weaknesses of its foundation models: VLM hallucination can inject erroneous predictions into the Curiosity Value Map, and although subtask feedback and quantitative scoring mitigate this, they cannot eliminate it. Performance scales strongly with VLM capability—the best open-source model tested, Qwen2.5-VL-7B (46.1% SR), trails Gemini 1.5 Pro (58.1% SR) by 12 points—creating cost, latency, and accessibility dependencies on commercial APIs. Each navigation step requires multiple VLM calls, limiting real-time operation. The evaluation is confined to indoor object-goal tasks in Habitat scenes; extension to real robots, dynamic environments, outdoor settings, and instruction-following tasks is untested.

6.2 Cross-Cutting Challenges

Model fidelity and compounding error. Every world model is an approximation, and errors compound when a model is rolled forward over long horizons. Planners and

policies can exploit model errors, choosing actions that look good in the model but fail in reality. DexSim2Real² shows that manipulation error grows with modelling error, and GAF shows that visually plausible predictions may not be physically feasible. Quantifying and propagating model uncertainty, and re-planning frequently with fresh observations, are essential mitigations.

The sim-to-real gap. Explicit and latent world models built or trained in simulation inherit the simulator’s simplifications: rigid bodies, idealized joints, simplified contacts, and clean sensing. Domain randomization, latency modelling, and real-to-sim reconstruction reduce but do not remove this gap, and deformable, granular, or fluid materials remain especially difficult.

Computation and latency. There is a persistent tension between model richness and decision speed. Sampling-based MPC over a physics engine takes minutes, VLM calls take seconds, and on-board locomotion must run at tens of hertz on embedded hardware. Selecting the right model for the task tempo, and distilling slow models into fast policies, is an open engineering problem.

Semantic grounding versus physical grounding. Geometric and physical world models understand shape and dynamics but not meaning; semantic world models understand meaning but not precise geometry or physics. Tasks such as “open the drawer that contains the scissors” require both, and no reviewed system provides both.

Reliability and hallucination of foundation models. As foundation models are increasingly used inside the control loop, their tendency to produce confident but wrong outputs becomes a safety concern. Structured outputs, quantitative scoring, cross-checking against observations, and geometric verification (as in WMNav’s distance-based stopping) are necessary safeguards.

Evaluation and benchmarking. The reviewed works use different robots, simulators, tasks, and metrics, making direct comparison impossible. Standardized benchmarks that evaluate world models across manipulation, locomotion, and navigation—including

prediction quality, downstream task success, and computational cost—would accelerate progress.

6.3 Future Research Directions

Building on the limitations above and on the future work proposed by the authors of the reviewed papers, several promising directions emerge:

- **Hybrid geometric–semantic world models:** enriching explicit and Gaussian world models with semantic priors from foundation models, so that a single representation supports both precise contact reasoning and language-level goals; GAF’s authors explicitly propose adding language modelling.
- **Mixed simulated and real training:** training latent world models on a mixture of simulated and real-world experience to reduce the sim-to-real gap, as suggested for WMP, and continually updating the model during deployment.
- **Multi-modal sensing:** incorporating tactile, force, and audio signals into world models to capture contact events and material properties that vision cannot observe, addressing failure modes observed in GAF and proposed for WMP.
- **Scene-level digital twins and mobile manipulation:** scaling from single-object twins to complete rooms and houses for long-horizon mobile manipulation, as proposed by the DexSim2Real² authors, and combining such twins with semantic navigation of the WMNav type.
- **Learning rewards from videos and language:** replacing hand-designed dense rewards with rewards generated from human videos or natural-language descriptions, reducing the engineering effort of MPC-based systems.
- **Efficient inference and distillation:** distilling slow planners and large VLMs into compact policies or smaller on-device models, enabling world-model reasoning at control rates on embedded hardware.
- **Uncertainty-aware and safe planning:** equipping world models with calibrated uncertainty so that planners can avoid regions where the model is unreliable and actively gather information when uncertainty is high, extending the interactive perception idea of DexSim2Real².

- **Unified benchmarks:** developing shared evaluation suites that measure world-model prediction accuracy and downstream embodied performance across domains.

6.4 Ethical and Societal Considerations

As world-model-driven robots move into homes, hospitals, and public spaces, their technical capabilities must be matched by attention to their wider impact. **Safety** is the foremost concern: a robot that plans inside an imperfect model can act confidently on wrong predictions, so safety constraints, conservative fall-back behaviours, and human oversight remain necessary even when the model is usually accurate. **Privacy** is a second concern: systems such as DexSim2Real² and WMNav continuously capture images of the private spaces in which they operate, and navigation agents that send panoramic views of a home to cloud-hosted VLMs raise questions about where that data is stored and who can access it. On-device models and data-minimizing designs are therefore desirable.

Accountability and transparency matter when a robot causes damage or fails at a task. World models that can be rendered or queried—digital twins, Gaussian renderings, and natural-language explanations—make it easier to reconstruct why a decision was taken, which supports debugging, certification, and user trust. Finally, **accessibility and sustainability** deserve attention: dependence on large proprietary models and powerful GPUs increases cost and energy consumption and may concentrate capability in a few organizations, whereas open, efficient models broaden access for researchers, students, and small companies.

6.5 Summary

In summary, world-model-based embodied AI faces challenges in model fidelity, sim-to-real alignment, semantic grounding, latency, and dependence on foundation-model quality. Addressing these issues requires hybrid training on simulated and real data, integration of tactile and force sensing, semantic priors within geometric world models, hallucination-resistant prompting and verification mechanisms, and efficient on-board inference. Progress along these directions will be essential to develop reliable, transparent, and broadly deployable world models for next-generation embodied intelligent systems.

Chapter 7

Conclusion

The integration of **world models** into **embodied AI** marks a major milestone in the evolution of intelligent robotics. Purely reactive policies, which map current observations directly to actions, often fail under the partial observability, complex dynamics, and open-endedness of the physical world. World-model-driven agents overcome these limitations by maintaining an internal, predictive representation of their environment—imagining the consequences of candidate actions before execution—thereby enabling precise planning, efficient exploration, and robust sim-to-real transfer across manipulation, locomotion, and navigation.

This study reviewed four key research contributions illustrating the diverse realizations of world models in embodied AI. **Paper [1]** (DexSim2Real²) constructed an *explicit* digital-twin world model of unseen articulated objects through interactive perception and 3D AIGC reconstruction, enabling sampling-based MPC with eigengrasp-reduced dexterous hands to achieve goal-conditioned manipulation errors of only a few degrees—without demonstrations or reinforcement learning—and generalizing to advanced skills such as tool use. **Paper [2]** (GAF) introduced a 4D Gaussian Action Field that unifies scene reconstruction, future prediction, and action extraction in a single feed-forward framework, improving average manipulation success by 7.3% over state-of-the-art baselines while running in real time from sparse unposed RGB views. **Paper [3]** (WMP) demonstrated that an RSSM world model trained entirely in simulation can serve as the perceptual backbone for visual legged locomotion, achieving the best reported traversal performance on a Unitree A1 quadruped and eliminating the teacher–student gap of privileged learning. Finally, **Paper [4]** (WMNav) integrated Vision-Language Models into a modular world model with an online Curiosity Value Map, attaining state-of-the-art zero-shot object goal

navigation on HM3D and MP3D without any task-specific training.

Taken together, these works trace a coherent design space—explicit physics-based, geometry-embedded, latent, and semantic world models—each optimizing a different balance among precision, data efficiency, latency, interpretability, and generality. The comparative analysis indicates that no single representation dominates; rather, the appropriate world model depends on the tempo and structure of the task at hand. The key lessons of this seminar can be summarized as follows:

- **Predicting before acting works across domains:** the same principle improves precision in manipulation, robustness in locomotion, and efficiency in navigation.
- **Priors substitute for data:** physics engines, 3D geometry, simulated experience, and internet-scale pre-training each reduce the amount of task-specific real-world data required.
- **Representation determines control strategy:** explicit simulators favour search, geometric fields favour registration and refinement, latent states favour learned policies, and semantic models favour prompting.
- **Interpretability is a practical asset:** renderable and inspectable models make it possible to verify plans before execution, an important property for safe deployment.

Future research is expected to converge these paradigms: enriching explicit and Gaussian world models with semantic priors from foundation models, training latent models on mixtures of simulated and real experience, incorporating tactile and force sensing, and scaling from single-object digital twins to scene-level world models for long-horizon mobile manipulation. Such convergence promises embodied agents that perceive, imagine, plan, and act with human-like foresight, bringing general-purpose robotic intelligence closer to practical reality.

References

- [1] T. Jiang, Y. Guan, L. Ma, J. Xu, J. Meng, W. Chen, Z. Zeng, L. Li, D. Wu, and R. Chen, “DexSim2Real²: Building Explicit World Model for Precise Articulated Object Dexterous Manipulation,” *IEEE Transactions on Robotics*, 2025.
- [2] Y. Chai, L. Deng, R. Shao, J. Zhang, K. Lv, L. Xing, X. Li, H. Zhang, and Y. Liu, “GAF: Gaussian Action Field as a 4D Representation for Dynamic World Modeling in Robotic Manipulation,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [3] H. Lai, J. Cao, J. Xu, H. Wu, Y. Lin, T. Kong, Y. Yu, and W. Zhang, “World Model-based Perception for Visual Legged Locomotion,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 11531–11537.
- [4] D. Nie, X. Guo, Y. Duan, R. Zhang, and L. Chen, “WMNav: Integrating Vision-Language Models into World Models for Object Goal Navigation,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 2392–2399.
- [5] D. Ha and J. Schmidhuber, “World models,” *arXiv preprint arXiv:1803.10122*, 2018.
- [6] R. S. Sutton, “Dyna, an integrated architecture for learning, planning, and reacting,” *ACM SIGART Bulletin*, vol. 2, no. 4, pp. 160–163, 1991.
- [7] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, “Learning latent dynamics for planning from pixels,” in *Proc. International Conference on Machine Learning (ICML)*, 2019, pp. 2555–2565.
- [8] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse domains through world models,” *arXiv preprint arXiv:2301.04104*, 2023.
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [10] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, and G. Martius, “Sample-efficient cross-entropy method for real-time planning,” in *Proc. Conference on Robot Learning (CoRL)*, PMLR vol. 155, 2021, pp. 1049–1065.
- [11] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.